ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	**INFORME DE PRÁCTICA DE LABORATORIO**

DEPARTAMENTO:	Ciencias de la Computación	**CARRERA:**	Ingeniería en Tecnologías de la Información		
ASIGNATURA:	Aplicaciones Distribuidas	**NIVEL:**	Séptimo	**FECHA:**	20/10/2026
DOCENTE:	Ing. Geovanny José Brito Casanova, Mgtr.	**PRÁCTICA N°:**	1	**CALIFICACIÓN:**	

Análisis de consistencia y disponibilidad de una aplicación distribuida en Kubernetes

Jhonny Joel Romero López

Tabla de contenidos

Resumen 8

Análisis de consistencia y disponibilidad de una aplicación distribuida en Kubernetes 9

Objetivos 9

Objetivo general 9

Objetivos específicos 9

Marco teórico 10

Desarrollo del laboratorio 12

Entorno de trabajo 12

Aplicación Node.js y contenedor Docker 14

Despliegue en Kubernetes: Deployment, Service NodePort y réplicas 16

Distribución de solicitudes y contadores locales independientes 18

Self-healing: eliminación y recuperación automática de un Pod 20

Escalamiento horizontal 23

Rolling Update con coexistencia temporal de versiones 24

Fallo con imagen inexistente v99 y rollback 26

Código fuente y configuraciones 29

Aplicación y contenedores 29

Manifiestos de Kubernetes 38

Scripts de prueba con k6 42

Coordinador del estado distribuido 48

Evidencias experimentales con k6 50

Parámetros y métricas de k6 50

Smoke Test 51
Baseline Test 52
Resilience Test 54
Update Test 55
Resultados de k6 y tabla comparativa 56
Configuración y volumen de solicitudes 56
Latencias y validaciones de las mismas corridas 57
Consistencia del estado distribuido 57
Alternativas consideradas 57
Implementación 58
Prueba de concurrencia 60
Reemplazo de una réplica y fallo del coordinador 62
Análisis de concurrencia, fallos y escenarios hipotéticos 65
Canary Deployment 66
Respuestas requeridas 69
Punto de degradación del sistema 70
Interpretación del punto de degradación 74
Análisis comparativo final 75
Respuestas al análisis 75
Conclusiones 77
Referencias 79
Apéndice A 80
Apéndice B 85

Índice de figuras

Figura 1. Verificación del entorno Node.js 12

Figura 2. Kubernetes habilitado en Docker Desktop 13

Figura 3. Instalación de k6 13

Figura 4. Creación del directorio del proyecto 14

Figura 5. Dockerfile de la aplicación 14

Figura 6. Aplicación v1 ejecutándose en un contenedor Docker 15

Figura 7. Contenedor en ejecución en Docker Desktop 16

Figura 8. Aplicación servida por Kubernetes mediante NodePort 16

Figura 9. Deployment con tres réplicas disponibles 17

Figura 10. Contenedores de los Pods gestionados por Kubernetes 18

Figura 11. Distribución de solicitudes entre Pods 19

Figura 12. Ruta /increment y contadores locales independientes por Pod 20

Figura 13. Eliminación manual de un Pod 21

Figura 14. Recuperación automática del Pod 22

Figura 15. Estado tras la recuperación 22

Figura 16. Escalamiento a diez réplicas 23

Figura 17. Los diez Pods en el nodo docker-desktop 24

Figura 18. Versión estable v2 antes del Rolling Update 25

Figura 19. Comparación del código en las imágenes v2 y v3 25

Figura 20. Coexistencia temporal de v2 y v3 durante el Rolling Update 26

Figura 21. Despliegue con la imagen incorrecta v99 27

Figura 22. El Deployment apuntando a v99 con los Pods antiguos activos 27

Figura 23. Persistencia del fallo del Pod v99 28

Figura 24. Rollback a la versión v3 28

Figura 25. Inicio del Smoke Test (1 VU, 10 s) 51

Figura 26. Resultados del Smoke Test 52

Figura 27. Inicio de una ejecución del Baseline con tres Pods 53

Figura 28. Resultados del Baseline con una réplica 54

Figura 29. Resultados del Resilience Test 55

Figura 30. Resultados del Update Test 56

Figura 31. Construcción de la imagen v4 para la consistencia 59

Figura 32. Coordinador central desplegado mediante Service interno 59

Figura 33. Tres réplicas v4 conectadas al coordinador, estado inicial en cero 60

Figura 34. Prueba de 100 incrementos concurrentes 61

Figura 35. Idempotencia: op-1 reenviada entre distintos Pods 61

Figura 36. Reemplazo de una réplica durante el experimento 62

Figura 37. Estado e idempotencia conservados tras el reemplazo 62

Figura 38. Fallo del coordinador: HTTP 503 63

Figura 39. Reinicio del coordinador: contador y operaciones en cero con un nuevo epoch 64

Figura 40. Pérdida del historial de idempotencia tras el reinicio 64

Figura 41. Grupos estable y Canary creados (4 + 1 réplicas) 67

Figura 42. Deployment original en cero, etiquetas track y Service NodePort 67

Figura 43. Ejecución del Canary Test 68

Figura 44. Resultados del Canary Test (reparto ≈ 78,8 % / 21,2 %) 69

Figura 45. Escenario con una réplica 71

Figura 46. Una réplica, 200 VUs: primer incumplimiento del umbral 71

Figura 47. Escenario con tres réplicas 72

Figura 48. Tres réplicas, 200 VUs: incumplimiento del umbral 72

Figura 49. Diez réplicas, 200 VUs: incumplimiento del umbral 73

Figura 50. Una réplica, 20 VUs 80

Figura 51. Una réplica, 50 VUs 81

Figura 52. Una réplica, 100 VUs 81

Figura 53. Tres réplicas, 20 VUs 82

Figura 54. Tres réplicas, 50 VUs 82

Figura 55. Tres réplicas, 100 VUs 83

Figura 56. Diez réplicas, 20 VUs 83

Figura 57. Diez réplicas, 50 VUs 84

Figura 58. Diez réplicas, 100 VUs 84

Figura 59. Interfaz de la aplicación v3 con panel de idempotencia 85

Figura 60. Código fuente de smoke-test.js 85

Figura 61. Corrida de carga de referencia (100 VUs, 30 s) 86

Figura 62. Fuente de baseline-test.js y segunda corrida 86

Figura 63. Coordinador escalado a cero y respuesta 503 (captura complementaria) 87

Figura 64. Regreso a tres réplicas para el experimento de consistencia 87

Figura 65. Verificación de sintaxis del coordinador 87

Figura 66. Verificación de sintaxis de la réplica v4 88

Figura 67. Imágenes locales tras construir v4 88

Figura 68. Cambio de imagen y rollback (resumen en PowerShell) 88

Figura 69. Entorno: winget 89

Índice de tablas

Tabla 1. Resumen de las pruebas k6 (valores reales de las capturas).....56

Tabla 2. Comparación de alternativas de consistencia 58

Tabla 3. Punto de degradación: 1, 3 y 10 réplicas × 20, 50, 100 y 200 VUs.....74

Tabla 4. Matriz comparativa de todos los experimentos 75

Resumen

Este laboratorio reproduce de manera individual, sobre un clúster local de Kubernetes (Docker Desktop, nodo docker-desktop, v1.34.1), el ciclo completo de una aplicación distribuida: desde su ejecución en un contenedor Docker hasta su despliegue con múltiples réplicas, y analiza su consistencia, disponibilidad, resiliencia y escalabilidad. Se construyó una aplicación Node.js con rutas /api/info, /increment y /health, se empaquetó en imágenes v1 a v4 y se expuso mediante un Service NodePort en el puerto 30080. Con la herramienta Grafana k6 se ejecutaron pruebas Smoke, Baseline, Resilience y Update, y se midieron VUs, solicitudes por segundo, errores y latencias (avg, p90, p95 y máximo). Se demostró la distribución de solicitudes entre Pods, los contadores locales independientes, la recuperación automática tras eliminar un Pod, el escalamiento horizontal, un Rolling Update con coexistencia temporal de v2 y v3, el fallo controlado con una imagen inexistente v99 y el rollback a v3. Como retos independientes se implementó un Canary Deployment (reparto aproximado 78,8 % v2 / 21,2 % v3, p95 de 13,41 ms y cero errores), un contador coordinado de estado distribuido con consistencia fuerte e idempotencia (100 incrementos concurrentes con contador final 100 y cero errores) y la determinación del punto de degradación, cuyo primer incumplimiento del umbral $p(95) < 250$ ms apareció a 200 VUs en los tres escenarios de 1, 3 y 10 réplicas. Los resultados confirman que Kubernetes mantiene la disponibilidad y recupera Pods automáticamente, pero que el estado en memoria no se replica ni persiste por sí solo, lo que obliga a una estrategia de coordinación explícita.

***Palabras clave**:* Kubernetes, consistencia distribuida, pruebas de carga con k6.

Análisis de consistencia y disponibilidad de una aplicación distribuida en Kubernetes

En el marco de la asignatura de Aplicaciones Distribuidas, este laboratorio integra las dos prácticas realizadas en clase sobre Node.js, Docker y Kubernetes, y las amplía con tres retos independientes de consistencia, despliegue Canary y análisis del punto de degradación. El objetivo no fue solo comprobar que los comandos funcionan, sino reproducir cada procedimiento documentando qué se hizo, para qué sirve, qué resultado se esperaba, qué ocurrió realmente y qué concepto de aplicaciones distribuidas demuestra.

El trabajo se desarrolló sobre un único nodo (docker-desktop), por lo que las conclusiones sobre capacidad deben leerse con esa limitación: diez Pods ejecutándose en el mismo nodo comparten la CPU, la memoria y la red del equipo y no equivalen a diez servidores físicos. A lo largo del informe se distingue de forma explícita lo que se probó de lo que es hipotético, y se conservan los resultados adversos observados (cuatro solicitudes fallidas en resiliencia y seis timeouts en la actualización) en lugar de presentarlos como ejecuciones perfectas.

Objetivos

Objetivo general

Analizar experimentalmente la consistencia, disponibilidad, resiliencia y escalabilidad de una aplicación distribuida desplegada en Kubernetes, reproduciendo el laboratorio desarrollado en clase y ejecutando pruebas independientes de concurrencia, carga, fallos, despliegues progresivos y coordinación del estado entre réplicas.

Objetivos específicos

- Desplegar una aplicación Node.js en Kubernetes con Deployment, Service NodePort, réplicas, etiquetas, imagePullPolicy y readiness probe.

- Evidenciar la distribución de solicitudes entre Pods, los contadores locales independientes, la recuperación automática tras eliminar un Pod y el escalamiento horizontal.
- Documentar un Rolling Update con coexistencia de versiones, un fallo controlado con una imagen inexistente y el rollback correspondiente.
- Ejecutar pruebas Smoke, Baseline, Resilience y Update con k6, interpretando sus métricas (VUs, duración, etapas, pausas, checks, thresholds, solicitudes/s, errores, avg, p90, p95 y máximo).
- Implementar y evaluar un Canary Deployment, un contador coordinado de estado distribuido y la determinación del punto de degradación del sistema.

Marco teórico

Contenedor e imagen. una imagen es una plantilla inmutable que empaqueta la aplicación y sus dependencias; un contenedor es una instancia en ejecución de esa imagen. El Dockerfile describe cómo se construye la imagen. La definición del Dockerfile se basa en Docker, Inc. (s. f.-a).

Pod, Deployment y réplicas. el Pod es la unidad mínima de ejecución en Kubernetes; el Deployment declara el estado deseado (qué imagen y cuántas réplicas) y un controlador mantiene ese número de Pods, recreándolos si fallan (self-healing).

Service y NodePort. un Service da una dirección estable y balancea el tráfico entre los Pods que coinciden con su selector. El tipo NodePort publica el servicio en un puerto del nodo (aquí el 30080). Las etiquetas del Pod y el selector del Service deben coincidir (The Kubernetes Authors, s. f.-c).

readinessProbe e imagePullPolicy. la readiness probe consulta /health para incorporar un Pod al Service solo cuando está listo; imagePullPolicy: Never obliga a usar la imagen local, sin descargarla de un registro. La prueba de readiness retira los endpoints no listos, pero no sustituye la coordinación del estado (The Kubernetes Authors, s. f.-a).

Escalamiento horizontal. consiste en aumentar o disminuir el número de réplicas (kubectl scale). Agrega capacidad de atención, pero no replica el estado que cada réplica guarda en su memoria.

Rolling Update, rollback y Canary. El Rolling Update sustituye los Pods gradualmente para mantener réplicas disponibles durante el cambio; ello no garantiza que todas las solicitudes individuales tengan éxito. El rollback recupera una revisión anterior. El Canary mantiene dos versiones y dirige una fracción del tráfico a la candidata (The Kubernetes Authors, s. f.-b).

Estado distribuido, consistencia e idempotencia. En esta solución, un único coordinador ordena las lecturas y escrituras del contador. La sección crítica no contiene esperas asincrónicas, por lo que una operación no observa un incremento parcial. La consistencia eventual, en cambio, permite divergencias temporales hasta que las copias converjan. La idempotencia evita aplicar dos veces una clave ya registrada durante la vida de ese coordinador; no implica persistencia.

k6 y sus métricas. k6 genera carga con usuarios virtuales (VUs). Las etapas (stages) controlan cómo suben y bajan los VUs; sleep es la pausa entre iteraciones; los checks validan respuestas y los thresholds definen criterios de aprobación. Las métricas clave son http_reqs y solicitudes/s, http_req_failed, y http_req_duration con su promedio, p90, p95 y máximo. Los criterios de aprobación se distinguen de las validaciones individuales (Grafana Labs, s. f.-a, s. f.-b).

Desarrollo del laboratorio

Esta sección reproduce, en orden cronológico, los procedimientos de las dos sesiones de clase. Cada procedimiento se cierra con una ficha de acción, propósito, resultado esperado, resultado obtenido, evidencia e interpretación, como exige la guía.

Entorno de trabajo

Se comprobó la versión del runtime con node -v, que devolvió v22.18.0. Node.js es el motor sobre el que corre la aplicación del laboratorio; confirmar su versión asegura la reproducibilidad del entorno antes de empaquetar la imagen.

Figura 1

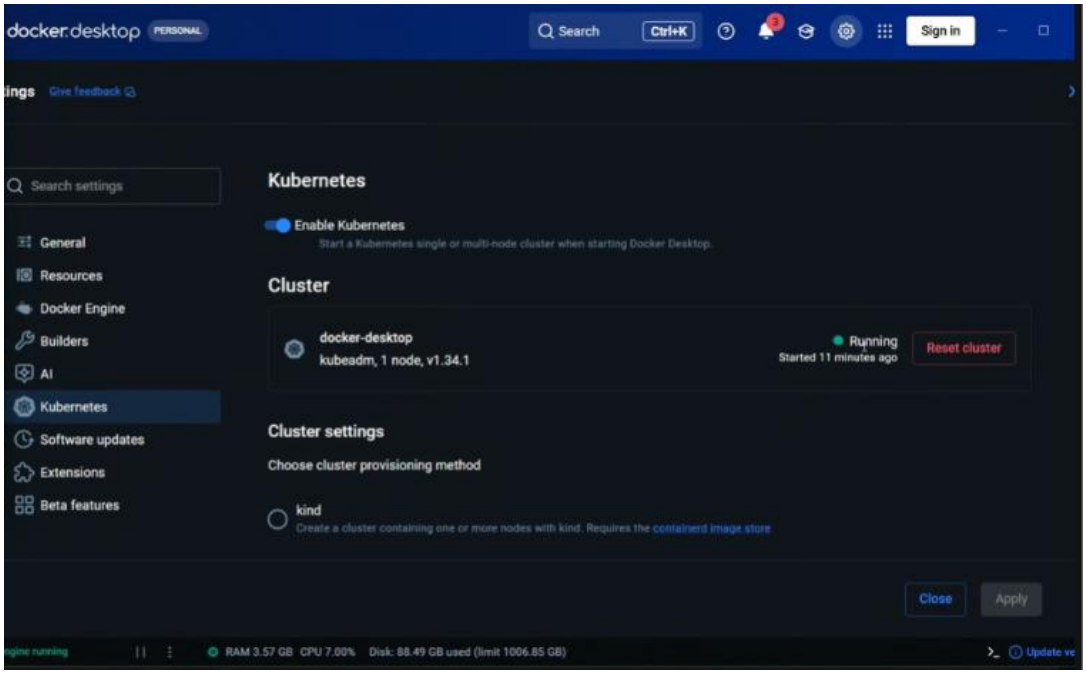
Verificación del entorno Node.js



Se activó Kubernetes en Docker Desktop: un clúster de un solo nodo llamado docker-desktop, versión v1.34.1, en estado Running. Este es el sustrato donde se desplegarán los Pods; al ser un único nodo, todas las réplicas comparten la CPU, la memoria y la red del equipo, dato clave para interpretar el punto de degradación. La configuración del entorno se documenta en Docker, Inc. (s. f.-b).

Figura 2

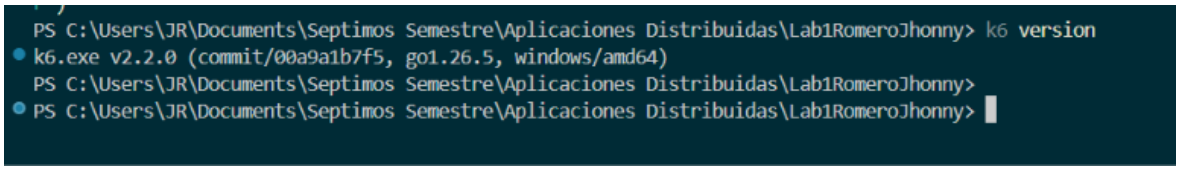
Kubernetes habilitado en Docker Desktop



Se verificó la versión de la herramienta de carga con k6 version, que reportó k6 v2.2.0 (windows/amd64). k6 es el generador de tráfico con el que se obtienen las métricas reproducibles de solicitudes, errores y latencia a lo largo del informe.

Figura 3

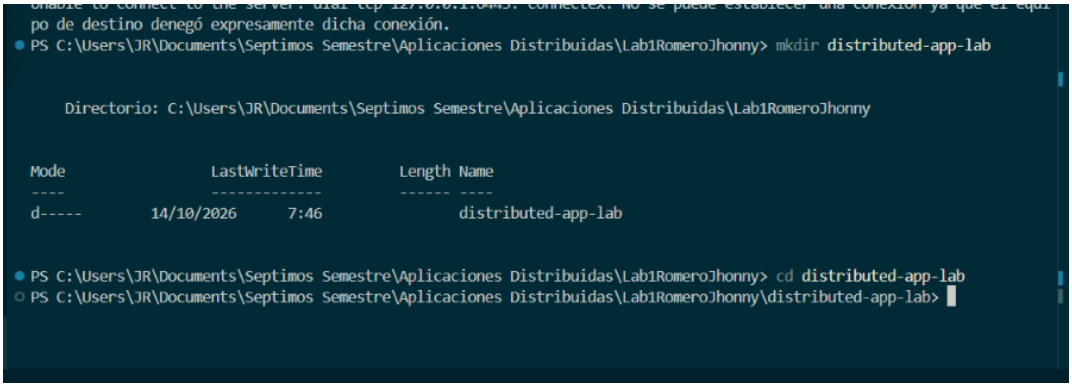
Instalación de k6



Se creó y se ingresó al directorio del proyecto (mkdir distributed-app-lab). Organizar el proyecto en una carpeta propia permite construir desde ahí la imagen y aplicar los manifiestos de Kubernetes de forma ordenada.

Figura 4

Creación del directorio del proyecto

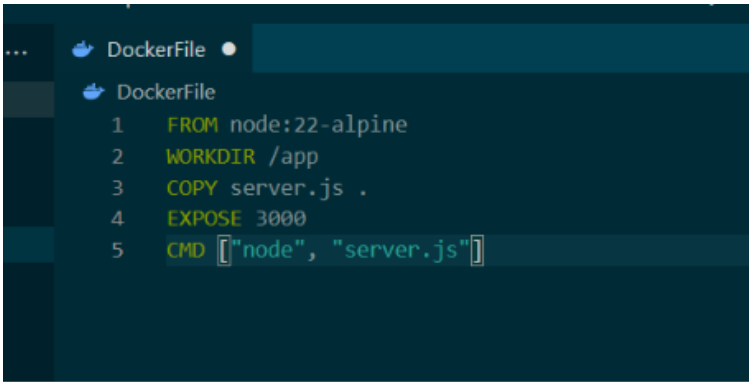


Aplicación Node.js y contenedor Docker

El Dockerfile parte de node:22-alpine, copia server.js al directorio /app, expone el puerto 3000 y arranca con node server.js. Describe de forma declarativa cómo se construye la imagen; la imagen es la plantilla inmutable y el contenedor es su instancia en ejecución.

Figura 5

Dockerfile de la aplicación

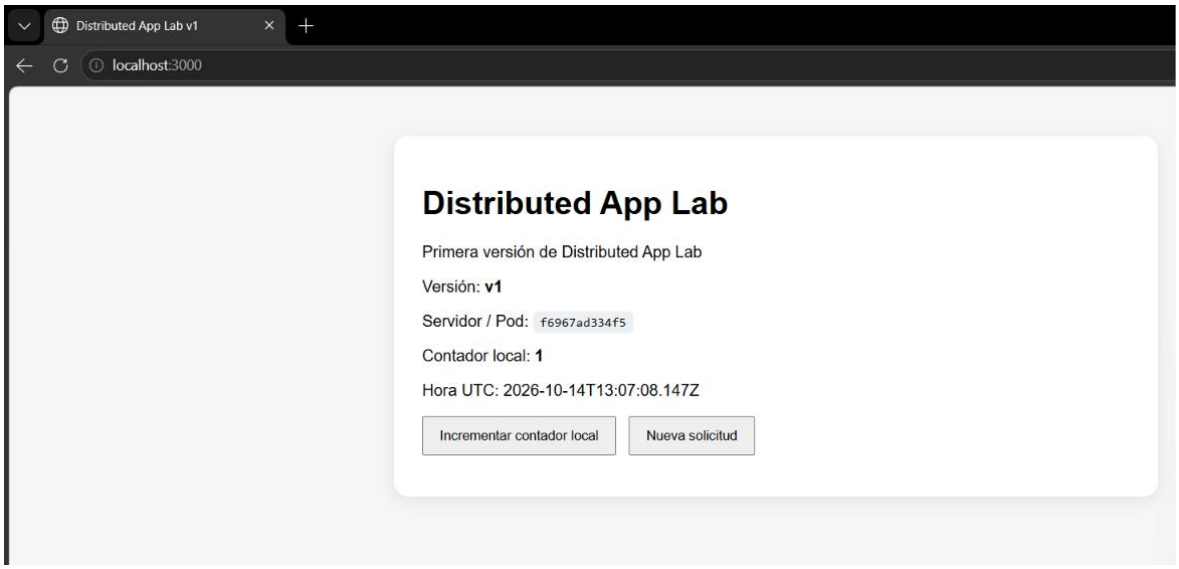


Al abrir localhost:3000 se observa la interfaz de la aplicación v1 servida directamente por el contenedor Docker, mostrando la versión, el Pod/servidor (f6967ad334f5), el contador local y

la hora UTC. Demuestra que la imagen se construyó y ejecuta correctamente como un único contenedor, antes de llevarla a Kubernetes.

Figura 6

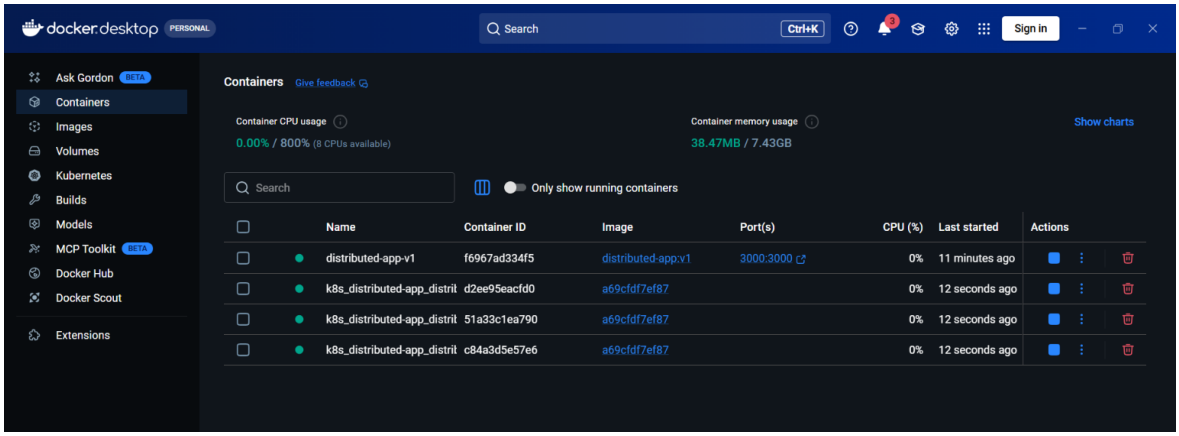
Aplicación v1 ejecutándose en un contenedor Docker



El panel de contenedores de Docker Desktop muestra el contenedor distributed-app-v1 (imagen distributed-app:v1, puerto 3000:3000) junto a los contenedores que Kubernetes crea para los Pods. Evidencia la diferencia entre un contenedor administrado manualmente por Docker y los que gestiona el orquestador.

Figura 7

Contenedor en ejecución en Docker Desktop

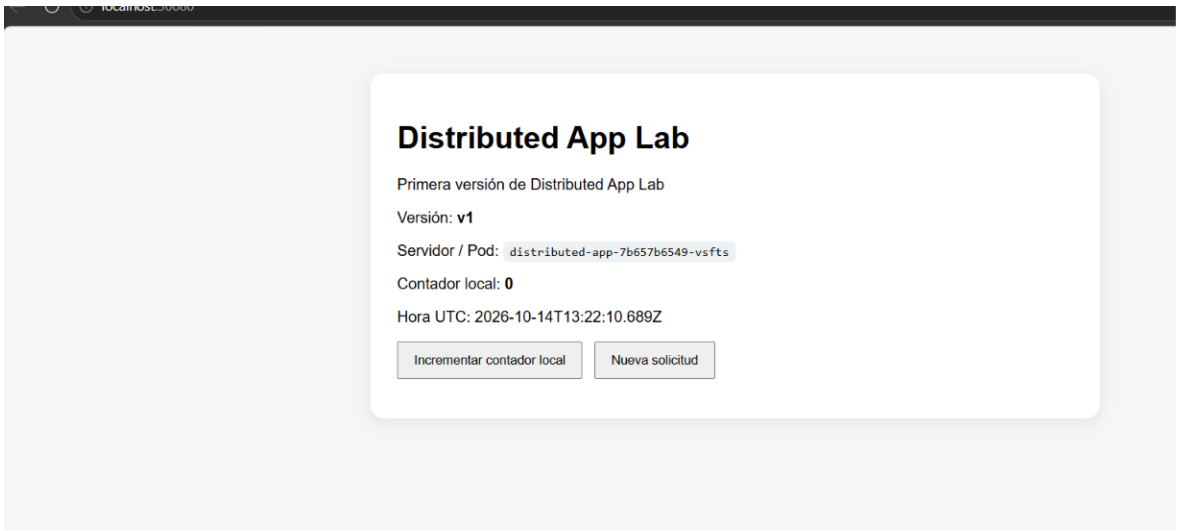


Despliegue en Kubernetes: Deployment, Service NodePort y réplicas

La misma aplicación, ahora detrás de un Service NodePort, responde en localhost:30080 y es atendida por un Pod gestionado por Kubernetes (distributed-app-7b657b6549-vsfts). El Service da una dirección estable y balancea el tráfico entre los Pods, desacoplando al cliente de cuál réplica responde.

Figura 8

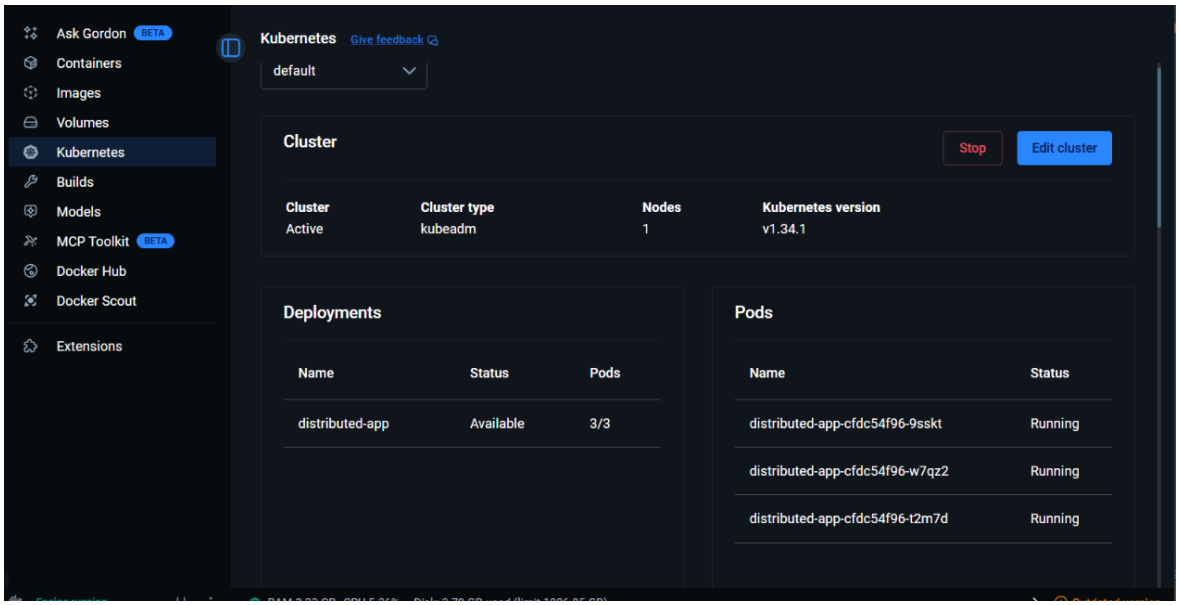
Aplicación servida por Kubernetes mediante NodePort



El panel de Kubernetes muestra el Deployment distributed-app como Available con 3/3 réplicas y los tres Pods en estado Running. El Deployment declara el estado deseado (imagen y número de réplicas) y el controlador lo mantiene; este es el mecanismo base de disponibilidad.

Figura 9

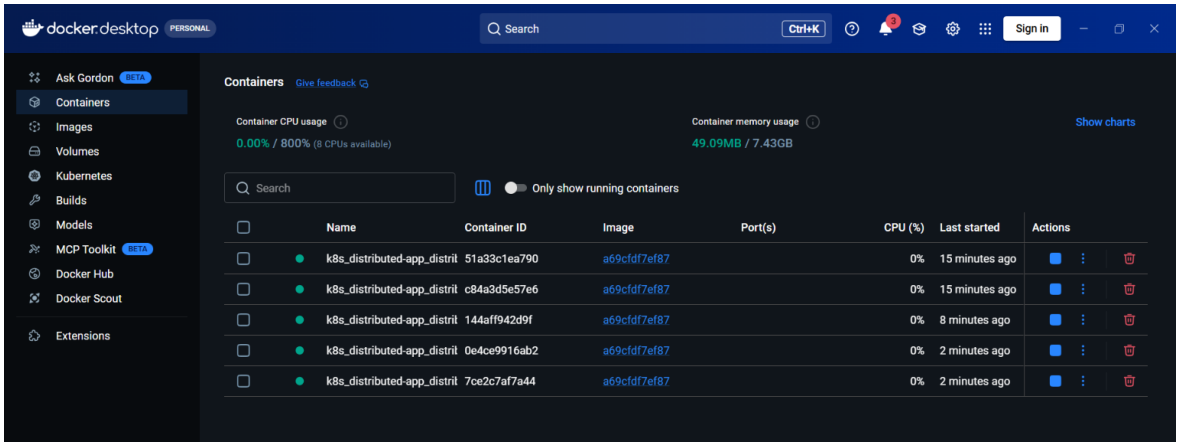
Deployment con tres réplicas disponibles



Docker Desktop lista los contenedores k8s_distributed-app correspondientes a cada réplica, todos a partir de la misma imagen. Confirma que cada Pod es una instancia independiente de la aplicación coordinada por Kubernetes, no un proceso compartido.

Figura 10

Contenedores de los Pods gestionados por Kubernetes

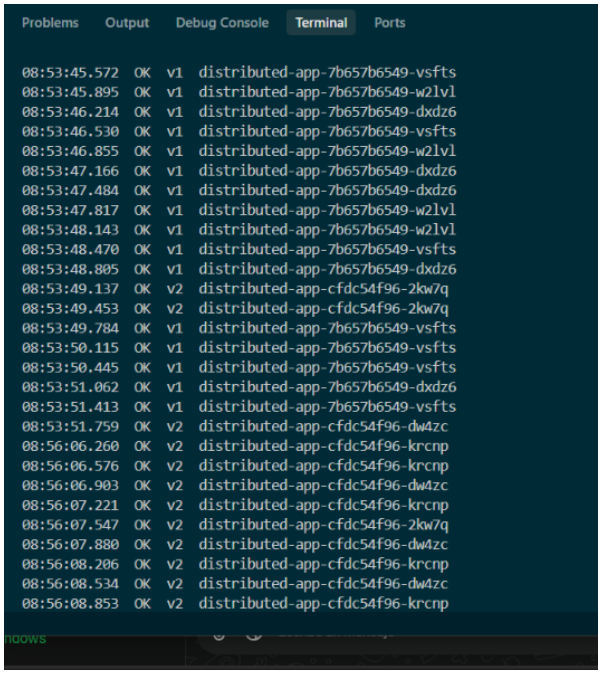


Distribución de solicitudes y contadores locales independientes

El registro muestra respuestas OK atendidas alternadamente por distintos Pods (y por versiones v1 y v2 durante un cambio). Evidencia que el Service reparte las solicitudes entre las réplicas disponibles: el cliente no sabe —ni necesita saber— qué Pod responde cada petición.

Figura 11

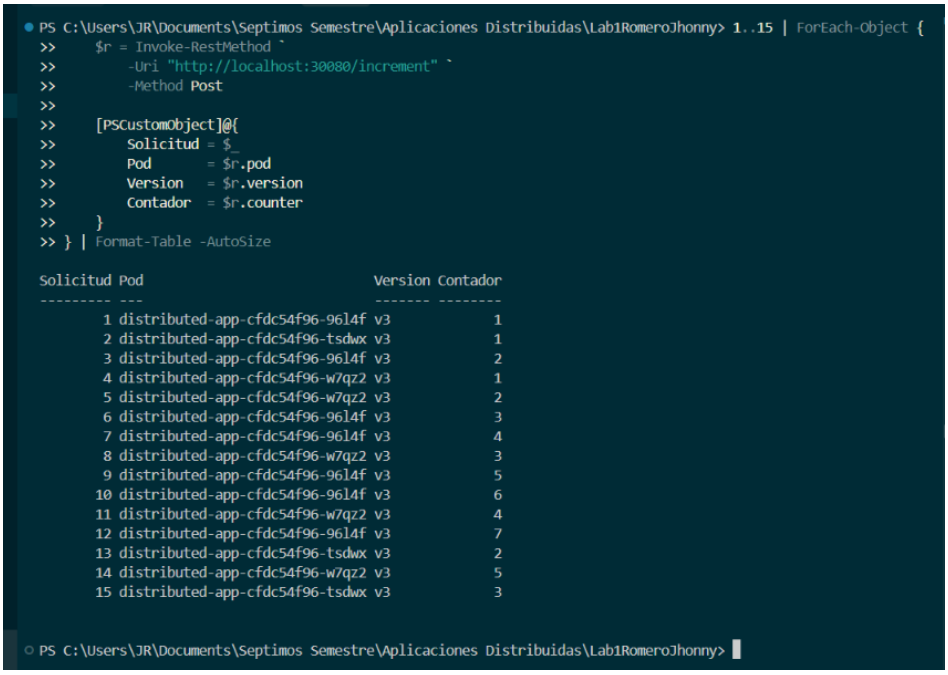
Distribución de solicitudes entre Pods



Al enviar quince POST a /increment, las respuestas provienen de tres Pods distintos (96l4f, w7qz2 y tsdwx) y cada uno lleva su propio contador (7, 5 y 3 incrementos). Demuestra el concepto de estado en memoria no replicado: cada réplica mantiene un contador local independiente, por lo que el mismo endpoint devuelve valores distintos según el Pod que atienda.

Figura 12

Ruta /increment y contadores locales independientes por Pod



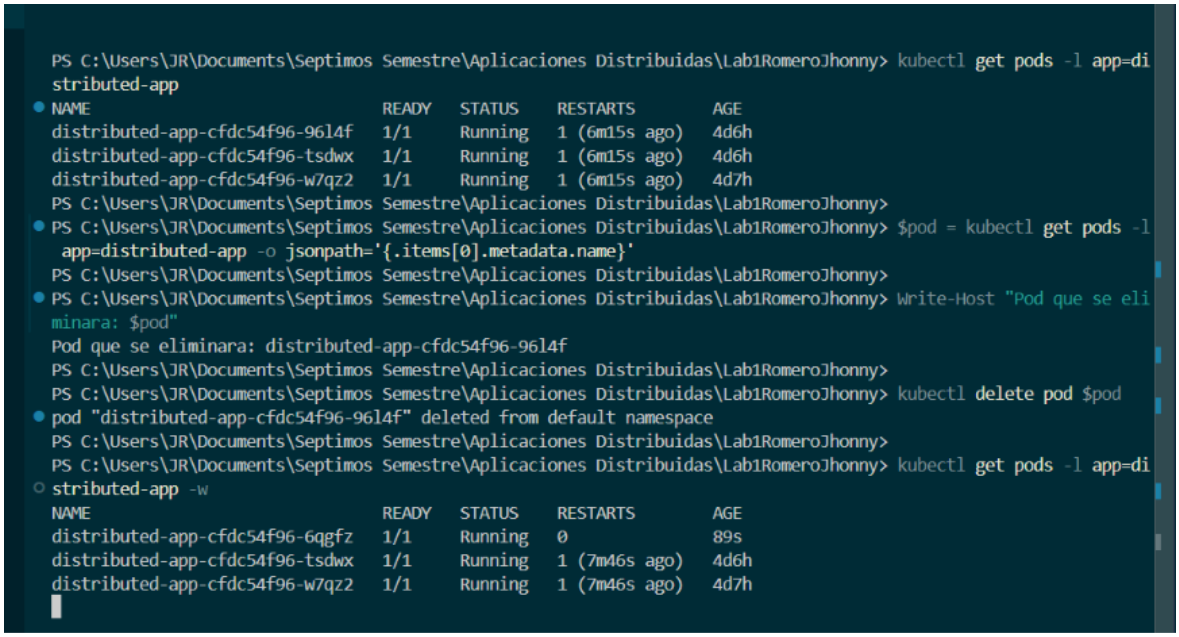
Acción realizada:	Se enviaron solicitudes repetidas a las rutas /api/info y /increment del Service NodePort (localhost:30080) con Invoke-RestMethod, registrando el Pod y la versión de cada respuesta.
Propósito:	Comprobar el balanceo del Service entre réplicas y observar el comportamiento del contador local de cada Pod.
Resultado esperado:	Las solicitudes deben ser atendidas por distintos Pods y cada Pod debe incrementar únicamente su propio contador.
Resultado obtenido:	Las respuestas se repartieron entre los tres Pods; los contadores locales llegaron a 7, 5 y 3 de forma independiente (Figura 12).
Evidencia:	Figuras 11 y 12.
Interpretación:	El estado guardado en memoria no se replica automáticamente entre réplicas: es un estado distribuido inconsistente que motiva el reto de consistencia del apartado «Consistencia del estado distribuido».

Self-healing: eliminación y recuperación automática de un Pod

Se seleccionó un Pod con kubectl y se eliminó con kubectl delete pod. Esta acción simula la caída de una réplica para observar cómo reacciona el controlador del Deployment.

Figura 13

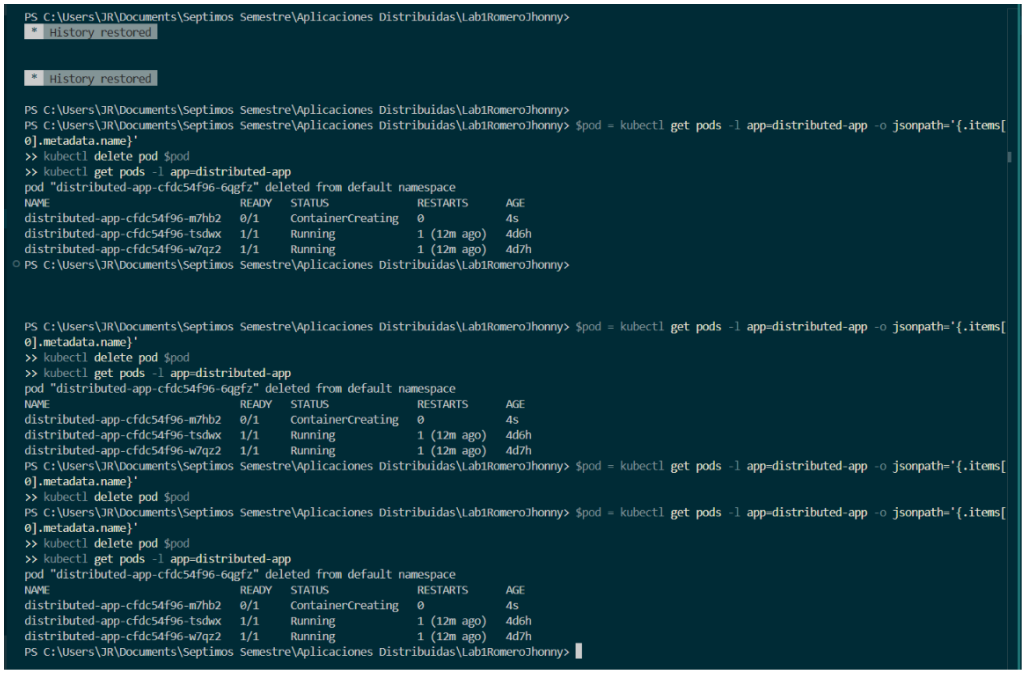
Eliminación manual de un Pod



Inmediatamente después de la eliminación, Kubernetes crea un Pod de reemplazo que aparece en estado ContainerCreating y luego Running, mientras los otros dos siguen atendiendo. Demuestra el self-healing: el controlador mantiene el número de réplicas declarado sin intervención manual.

Figura 14

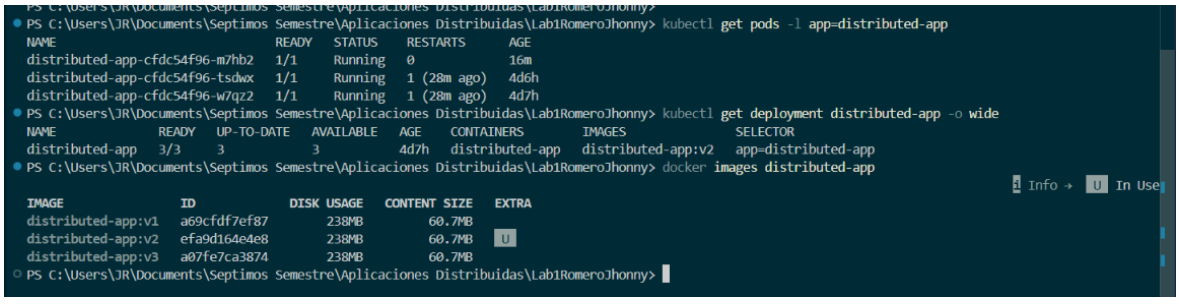
Recuperación automática del Pod



La consulta posterior muestra el Deployment nuevamente en 3/3 réplicas Running con la imagen en uso (v2). Confirma que, tras el reemplazo, el sistema vuelve al estado deseado y la disponibilidad se mantiene.

Figura 15

Estado tras la recuperación



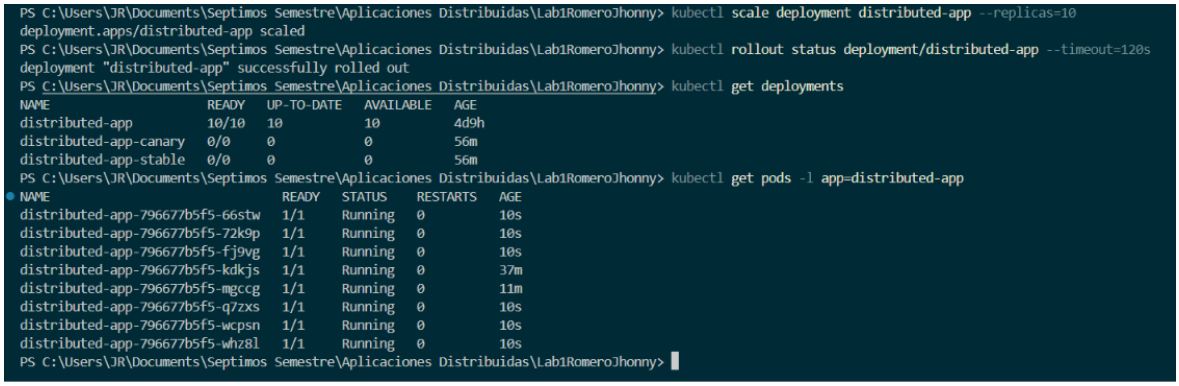
Acción realizada:	Se eliminó deliberadamente un Pod (kubectl delete pod) y se observó la lista de Pods con la bandera -w.
Propósito:	Evidenciar la recuperación automática (self-healing) del Deployment.
Resultado esperado:	Kubernetes debe crear un nuevo Pod automáticamente para volver a las tres réplicas declaradas.
Resultado obtenido:	Apareció un Pod nuevo (ContainerCreating → Running) y el Deployment regresó a 3/3 (Figuras 13 a 15).
Evidencia:	Figuras 13, 14 y 15.
Interpretación:	El Deployment garantiza disponibilidad a nivel de réplicas, pero el Pod nuevo nace con su contador local en cero: la disponibilidad del proceso no implica la preservación del estado.

Escalamiento horizontal

Con kubectl scale --replicas=10 el Deployment pasó a 10/10 réplicas disponibles y diez Pods Running. El escalamiento horizontal agrega capacidad de atención concurrente, pero no modifica ni unifica el estado que cada réplica mantiene en memoria.

Figura 16

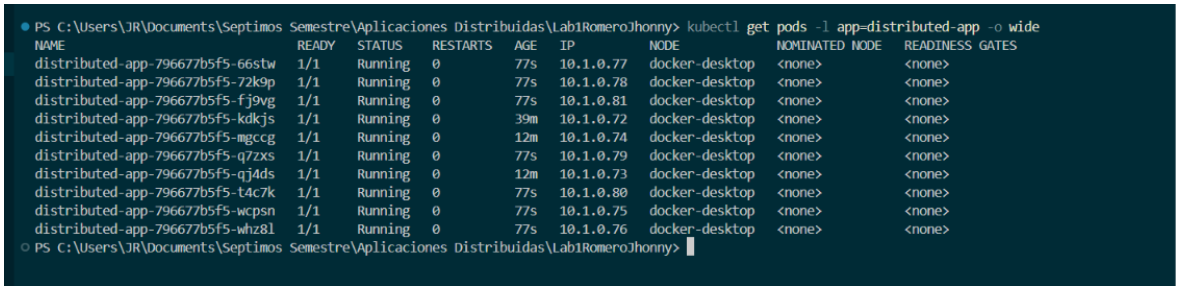
Escalamiento a diez réplicas



El listado con -o wide muestra los diez Pods con IP propia, todos en el mismo NODE docker-desktop. Es la evidencia central para interpretar el punto de degradación: diez Pods en un único nodo comparten la CPU y la memoria del equipo y no equivalen a diez servidores físicos.

Figura 17

Los diez Pods en el nodo docker-desktop



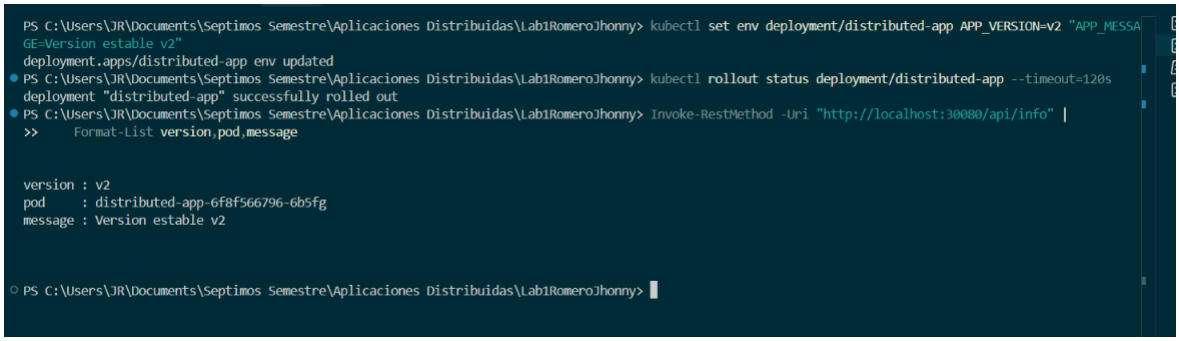
Acción realizada:	Se escaló el Deployment a 1, 3 y 10 réplicas con kubectl scale y se verificó el estado con kubectl get pods -o wide.
Propósito:	Observar qué cambia y qué no cambia al escalar horizontalmente.
Resultado esperado:	El número de Pods disponibles debe variar según las réplicas solicitadas.
Resultado obtenido:	Se alcanzaron 1/1, 3/3 y 10/10 réplicas; todos los Pods quedaron en el nodo docker-desktop (Figuras 16 y 17).
Evidencia:	Figuras 16 y 17 (y Figura 45 del apartado «Punto de degradación del sistema»).
Interpretación:	Escalar aumenta la capacidad de atención concurrente, pero el estado local de cada réplica sigue siendo independiente; además, al compartir un solo nodo, el beneficio del escalamiento está acotado por los recursos del equipo.

Rolling Update con coexistencia temporal de versiones

Antes de actualizar, se fijó la versión estable v2 (kubectl set env) y la consulta a /api/info confirmó version v2 y el mensaje “Version estable v2”. Establece el punto de partida conocido sobre el que se aplicará la actualización progresiva.

Figura 18

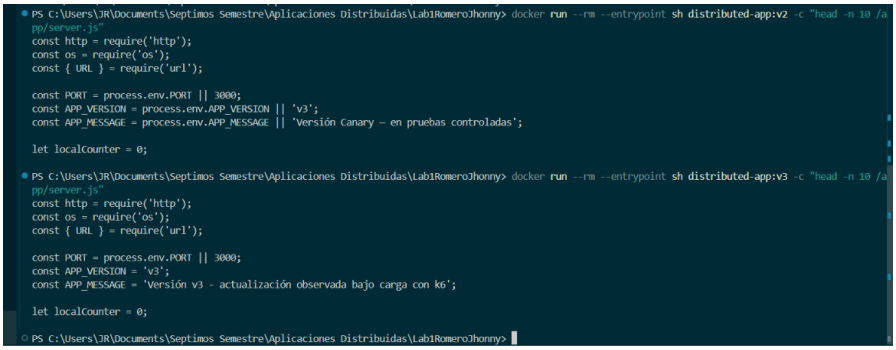
Versión estable v2 antes del Rolling Update



Inspeccionando las primeras líneas de server.js dentro de cada imagen se observa que v2 y v3 difieren en APP_VERSION y APP_MESSAGE. Confirma que son dos versiones reales y distintas de la aplicación, condición necesaria para demostrar el Rolling Update.

Figura 19

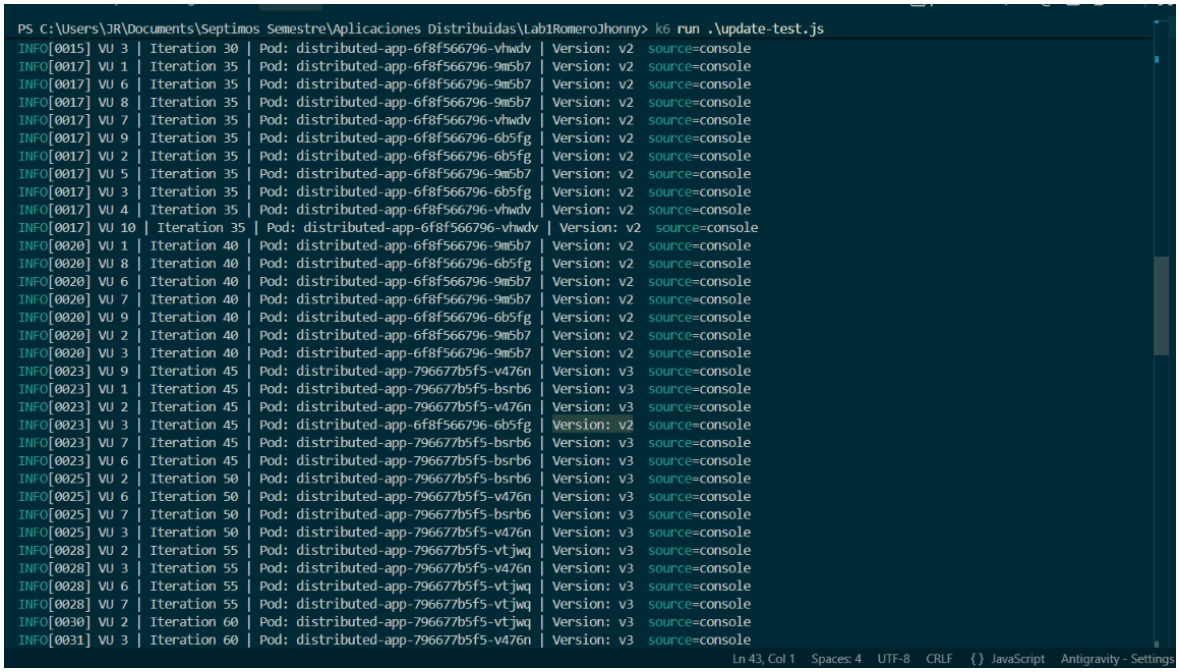
Comparación del código en las imágenes v2 y v3



Durante el cambio de imagen a v3, el registro de k6 muestra que, alrededor del segundo 23, conviven respuestas de Pods v2 y v3 simultáneamente. Es la firma del Rolling Update: Kubernetes reemplaza los Pods de forma gradual. El Deployment base no define un bloque strategy, por lo que aplica la estrategia RollingUpdate por defecto (maxUnavailable y maxSurge del 25 %); aun así, mantiene Pods disponibles durante la transición.

Figura 20

Coexistencia temporal de v2 y v3 durante el Rolling Update



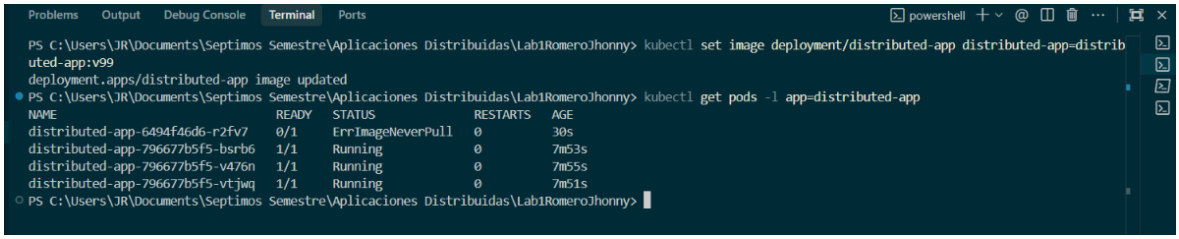
Acción realizada:	Se aplicó kubectl set image distributed-app=distributed-app:v3 con la aplicación en ejecución, mientras k6 generaba tráfico (update-test.js).
Propósito:	Actualizar la versión sin interrumpir el servicio y observar la transición.
Resultado esperado:	Las peticiones deben seguir atendiéndose mientras los Pods v2 se reemplazan gradualmente por Pods v3.
Resultado obtenido:	Hubo coexistencia temporal de v2 y v3 (Figura 20); el servicio siguió respondiendo, con seis timeouts puntuales de 731 solicitudes (apartado «Update Test»).
Evidencia:	Figuras 18, 19 y 20; Figura 30 (resultados k6).
Interpretación:	El Rolling Update ofrece disponibilidad continua durante el despliegue, aunque pueden aparecer errores transitorios mientras un Pod se da de baja y otro toma su lugar.

Fallo con imagen inexistente v99 y rollback

Al apuntar el Deployment a una imagen inexistente (distributed-app:v99), el nuevo Pod queda en estado ErrImageNeverPull —no ErrImagePull— porque imagePullPolicy es Never y la imagen no existe localmente. Reproduce un despliegue defectuoso de forma controlada.

Figura 21

Despliegue con la imagen incorrecta v99



La consulta del Deployment muestra IMAGES distributed-app:v99 con UP-TO-DATE 1 pero AVAILABLE 3: el Pod nuevo nunca llega a estar listo, mientras los tres Pods anteriores siguen atendiendo. Demuestra que el Rolling Update protege la disponibilidad al no retirar los Pods viejos hasta que el nuevo esté disponible (Running), algo que el Pod de v99 nunca alcanzó.

Figura 22

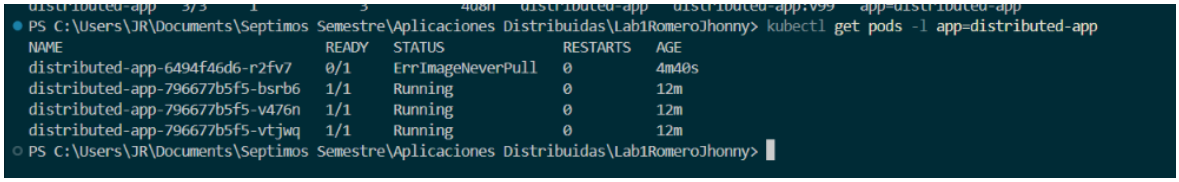
El Deployment apuntando a v99 con los Pods antiguos activos



Minutos después, el Pod de v99 sigue en ErrImageNeverPull: la imagen local no existe y el contenedor no puede arrancar. Con tres réplicas y la estrategia RollingUpdate por defecto, maxUnavailable=25 % se redondea a cero y maxSurge=25 % a uno; por ello los tres Pods anteriores permanecen disponibles. En esta etapa no había readinessProbe explícita: la prueba /health se añadió en v4 (The Kubernetes Authors, s. f.-b).

Figura 23

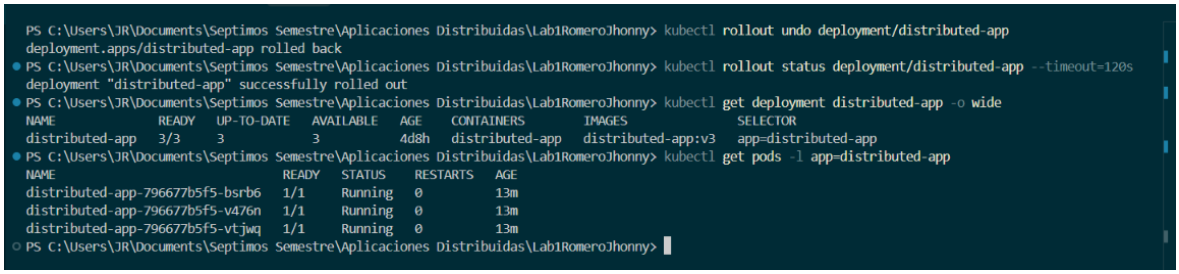
Persistencia del fallo del Pod v99



Con kubectl rollout undo el Deployment vuelve a la imagen v3, queda en 3/3 réplicas Running y el despliegue se marca como exitoso. Demuestra la recuperación ante un despliegue incorrecto mediante el historial de revisiones de Kubernetes.

Figura 24

Rollback a la versión v3



Acción realizada:	Se forzó un despliegue con una imagen inexistente (kubectl set image ... :v99) y luego se deshizo con kubectl rollout undo.
Propósito:	Documentar la coexistencia de versiones ante un despliegue incorrecto y la recuperación mediante rollback.
Resultado esperado:	El Pod nuevo no debe estar listo y el servicio debe seguir atendido por los Pods anteriores; el rollback debe restaurar una versión sana.
Resultado obtenido:	El Pod de v99 quedó en ErrImageNeverPull con AVAILABLE 3; el rollback restauró v3 en 3/3 (Figuras 21 a 24).
Evidencia:	Figuras 21, 22, 23 y 24.
Interpretación:	El Pod de v99 no pudo arrancar porque la imagen no existía localmente (ErrImageNeverPull, con imagePullPolicy: Never); no se trató de un fallo de readiness —el Deployment base ni siquiera define una readinessProbe—. Como el Rolling Update no retira los Pods antiguos hasta que el nuevo esté disponible, el servicio siguió atendido por los tres Pods de v3, y el historial de revisiones permitió revertir con un solo comando.

Código fuente y configuraciones

A continuación se presentan, con formato de código diferenciado, los archivos reales utilizados en el laboratorio.

Aplicación y contenedores

server.js local con contador independiente versión v3

Este archivo conserva el contador en la memoria de cada proceso. GET /api/info informa versión, Pod y contador; POST /increment aumenta solo ese contador y GET /health responde OK. El servidor usa el módulo HTTP nativo de Node.js (Node.js, s. f.).

```
const http = require('http');
const os = require('os');
const { URL } = require('url');

const PORT = process.env.PORT || 3000;
const APP_VERSION = 'v3';
const APP_MESSAGE = 'Versión v3 - actualización observada bajo carga con k6';

let localCounter = 0;

function commonHeaders(contentType = 'text/html; charset=utf-8') {
    return {
        'Content-Type': contentType,
        'Cache-Control': 'no-store',
        'Connection': 'close'
    };
}
```

```
function info() {
    return {
        version: APP_VERSION,
        message: APP_MESSAGE,
        pod: os.hostname(),
        counter: localCounter,
        time: new Date().toISOString()
    };
}

const server = http.createServer((req, res) => {
    const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`);

    if (req.method === 'GET' && url.pathname === '/health') {
        res.writeHead(200, commonHeaders('text/plain; charset=utf-8'));
        return res.end('OK');
    }

    if (req.method === 'GET' && url.pathname === '/api/info') {
        res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
        return res.end(JSON.stringify(info()));
    }

    if (req.method === 'POST' && url.pathname === '/increment') {
        localCounter++;
        res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
        return res.end(JSON.stringify({ action: 'increment', ...info() }));
    }

    if (req.method === 'GET' && url.pathname === '/') {
        const data = info();
        const html = `<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Distributed App Lab ${data.version}</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 760px; margin: 50px auto; padding: 0
20px; background: #f4f6f8; }
    .card { background: white; border-radius: 14px; padding: 28px; box-shadow: 0 4px 16px
rgba(0,0,0,.08); }
    code { background: #eef1f4; padding: 3px 7px; border-radius: 6px; }
    button { padding: 10px 16px; margin-right: 8px; cursor: pointer; }
    .value { font-weight: bold; }
    #result { white-space: pre-wrap; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Distributed App Lab</h1>
    <p>${data.message}</p>
    <p>Versión: <span class="value">${data.version}</span></p>
    <p>Servidor / Pod: <code>${data.pod}</code></p>
    <p>Contador local: <span class="value">${data.counter}</span></p>
    <p>Hora UTC: ${data.time}</p>
    <button onclick="incrementar()">Incrementar contador local</button>
    <button onclick="location.reload()">Nueva solicitud</button>
    <pre id="result"></pre>
  </div>
```

```
  <script>
    async function incrementar() {
      const response = await fetch('/increment', { method: 'POST' });
      const data = await response.json();
      document.getElementById('result').textContent = JSON.stringify(data, null, 2);
    }
  </script>
</body>
</html>`;

        res.writeHead(200, commonHeaders());
        return res.end(html);
    }

    res.writeHead(404, commonHeaders('application/json; charset=utf-8'));
    res.end(JSON.stringify({ error: 'Not found' }));
});

server.listen(PORT, '0.0.0.0', () => {
    console.log(`Distributed App Lab ${APP_VERSION} listening on port ${PORT}`);
    console.log(`Hostname: ${os.hostname()}`);
});

process.on('SIGTERM', () => {
    console.log('SIGTERM recibido. Cerrando servidor de forma controlada...');
    server.close(() => {
        console.log('Servidor cerrado.');
        process.exit(0);
    });

    setTimeout(() => process.exit(1), 10000).unref();
});
```

Variante configurable de la aplicación local

Archivo distributed-app-lab/server.js conservado en el proyecto. APP_VERSION y APP_MESSAGE admiten variables de entorno. Su contador y su mapa de pagos son locales a cada proceso: el panel de idempotencia de la Figura 59 no demuestra deduplicación compartida entre Pods. La deduplicación global probada corresponde al coordinador v4 del apartado «Consistencia del estado distribuido».

```
const http = require('http');
const os = require('os');
const { URL } = require('url');

const PORT = process.env.PORT || 3000;
const APP_VERSION = process.env.APP_VERSION || 'v3';
const APP_MESSAGE = process.env.APP_MESSAGE || 'Versión Canary — en pruebas controladas';

let localCounter = 0;

// ─── Simulación de almacenamiento compartido de pagos (idempotencia) ───
// En producción esto sería Redis o una base de datos compartida.
// Aquí usamos un Map en memoria para demostrar el concepto.
const processedPayments = new Map();
```

```
function commonHeaders(contentType = 'text/html; charset=utf-8') {
  return {
    'Content-Type': contentType,
    'Cache-Control': 'no-store',
    'Connection': 'close'
  };
}

function info() {
  return {
    version: APP_VERSION,
    message: APP_MESSAGE,
    pod: os.hostname(),
    counter: localCounter,
    time: new Date().toISOString()
  };
}

const server = http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`);

  // ─── Health Check ───
  if (req.method === 'GET' && url.pathname === '/health') {
    res.writeHead(200, commonHeaders('text/plain; charset=utf-8'));
    return res.end('OK');
  }

  // ─── Info API ───
  if (req.method === 'GET' && url.pathname === '/api/info') {
    res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
    return res.end(JSON.stringify(info()));
  }

  // ─── Incrementar contador local ───
  if (req.method === 'POST' && url.pathname === '/increment') {
    localCounter++;
    res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
    return res.end(JSON.stringify({ action: 'increment', ...info() }));
  }

  // ─── ENDPOINT DE PAGO IDEMPOTENTE ───
  // Demuestra cómo evitar pagos duplicados usando una clave de idempotencia.
  // Cada solicitud de pago lleva un "idempotencyKey" único.
  // Si el mismo key llega a cualquier pod, solo se procesa la primera vez.
  if (req.method === 'POST' && url.pathname === '/api/payment') {
    let body = '';
    req.on('data', chunk => { body += chunk; });
    req.on('end', () => {
      try {
        const payment = JSON.parse(body);
        const idempotencyKey = payment.idempotencyKey;

        if (!idempotencyKey) {
          res.writeHead(400, commonHeaders('application/json; charset=utf-8'));
          return res.end(JSON.stringify({
            error: 'Falta idempotencyKey',
            explanation: 'Cada pago debe incluir una clave única para evitar duplicados'
          }));
        }
```

```
        // Verificar si ya fue procesado
        if (processedPayments.has(idempotencyKey)) {
          const original = processedPayments.get(idempotencyKey);
          res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
          return res.end(JSON.stringify({
            status: 'DUPLICADO — ya fue procesado',
            originalResult: original,
            processedBy: original.processedBy,
            currentPod: os.hostname(),
            explanation: 'Este pago ya fue confirmado. No se volvió a cobrar.'
          }));
        }

        // Procesar el pago (simulación)
        const result = {
          status: 'APROBADO',
          idempotencyKey,
          amount: payment.amount,
          currency: payment.currency || 'USD',
          processedBy: os.hostname(),
          processedAt: new Date().toISOString(),
          transactionId: `TXN-${Date.now()}-${Math.random().toString(36).substr(2, 6)}`
        };

        // Guardar en el registro de pagos procesados
        processedPayments.set(idempotencyKey, result);

        res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
        return res.end(JSON.stringify(result));

      } catch (e) {
        res.writeHead(400, commonHeaders('application/json; charset=utf-8'));
        return res.end(JSON.stringify({ error: 'JSON inválido', details: e.message }));
      }
    });
    return;
  }

  // ─── Listar pagos procesados (para demostración) ───
  if (req.method === 'GET' && url.pathname === '/api/payments') {
    const payments = Array.from(processedPayments.entries()).map(([key, val]) => ({
idempotencyKey: key, ...val }));
    res.writeHead(200, commonHeaders('application/json; charset=utf-8'));
    return res.end(JSON.stringify({ pod: os.hostname(), totalPayments: payments.length,
payments }));
  }

  // ─── Página principal con UI de pruebas ───
  if (req.method === 'GET' && url.pathname === '/') {
    const data = info();
    const versionColor = data.version === 'v3' ? '#e74c3c' : data.version === 'v2' ? '#3498db'
: '#27ae60';
    const html = `<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Distributed App Lab ${data.version}</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
```

```
    body { font-family: 'Segoe UI', Arial, sans-serif; max-width: 820px; margin: 40px auto;
padding: 0 20px; background: #f0f2f5; color: #333; }
    .card { background: white; border-radius: 16px; padding: 28px; box-shadow: 0 4px 20px
rgba(0,0,0,.06); margin-bottom: 20px; }
    .version-badge { display: inline-block; background: ${versionColor}; color: white;
padding: 4px 14px; border-radius: 20px; font-weight: bold; font-size: 14px; }
    h1 { margin-bottom: 8px; }
    h2 { margin: 18px 0 10px; color: #555; font-size: 18px; }
    code { background: #eef1f4; padding: 3px 7px; border-radius: 6px; font-size: 13px; }
    button { padding: 10px 18px; margin: 4px 4px 4px 0; cursor: pointer; border: none; border-
radius: 8px; font-size: 14px; color: white; transition: transform .1s; }
    button:hover { transform: scale(1.03); }
    button:active { transform: scale(0.97); }
    .btn-blue { background: #3498db; }
    .btn-green { background: #27ae60; }
    .btn-orange { background: #e67e22; }
    .btn-red { background: #e74c3c; }
    .info-row { display: flex; justify-content: space-between; padding: 6px 0; border-bottom:
1px solid #f0f0f0; }
    .info-row:last-child { border-bottom: none; }
    .label { color: #888; }
    .value { font-weight: bold; }
    pre { background: #1e1e2e; color: #a6e3a1; padding: 16px; border-radius: 10px; overflow-x:
auto; font-size: 13px; white-space: pre-wrap; margin-top: 10px; min-height: 60px; }
    .section-title { font-size: 15px; color: #666; margin-bottom: 8px; text-transform:
uppercase; letter-spacing: 1px; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Distributed App Lab <span class="version-badge">${data.version}</span></h1>
    <p style="margin: 8px 0 16px; color: #666;">${data.message}</p>
    <div class="info-row"><span class="label">Servidor /
Pod</span><code>${data.pod}</code></div>
    <div class="info-row"><span class="label">Contador local</span><span
class="value">${data.counter}</span></div>
    <div class="info-row"><span class="label">Hora UTC</span><span>${data.time}</span></div>
  </div>

  <div class="card">
    <p class="section-title">Acciones</p>
    <button class="btn-blue" onclick="incrementar()">Incrementar Contador</button>
    <button class="btn-green" onclick="location.reload()">Refrescar</button>
  </div>

  <div class="card">
    <p class="section-title">Prueba de Pago Idempotente</p>
    <p style="color:#666; font-size:13px; margin-bottom:12px;">
      Demuestra cómo evitar pagos duplicados. Cada pago lleva una <b>clave de idempotencia</b>
única.
      Si haces clic dos veces con la misma clave, solo se cobra una vez.
    </p>
    <button class="btn-green" onclick="enviarPago()">Enviar Pago ($25.00)</button>
    <button class="btn-orange" onclick="enviarPago()">Reenviar Mismo Pago</button>
    <button class="btn-red" onclick="nuevoPago()">Nuevo Pago (Nueva Clave)</button>
    <button class="btn-blue" onclick="verPagos()">Ver Pagos del Pod</button>
    <p style="margin-top:8px; font-size:12px; color:#999;">Clave actual: <code
id="currentKey">—</code></p>
  </div>

  <div class="card">
```

```
    <p class="section-title">Resultado</p>
    <pre id="result">Haz clic en un botón para ver el resultado aquí...</pre>
  </div>

  <script>
    let currentIdempotencyKey = crypto.randomUUID();
    document.getElementById('currentKey').textContent = currentIdempotencyKey;

    async function incrementar() {
      const r = await fetch('/increment', { method: 'POST' });
      const data = await r.json();
      document.getElementById('result').textContent = JSON.stringify(data, null, 2);
    }

    async function enviarPago() {
      const r = await fetch('/api/payment', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          idempotencyKey: currentIdempotencyKey,
          amount: 25.00,
          currency: 'USD',
          description: 'Pago de prueba del laboratorio'
        })
      });
      const data = await r.json();
      document.getElementById('result').textContent = JSON.stringify(data, null, 2);
    }

    function nuevoPago() {
      currentIdempotencyKey = crypto.randomUUID();
      document.getElementById('currentKey').textContent = currentIdempotencyKey;
      document.getElementById('result').textContent = 'Nueva clave generada: ' +
currentIdempotencyKey + '\\nAhora haz clic en "Enviar Pago" para procesar.';
    }

    async function verPagos() {
      const r = await fetch('/api/payments');
      const data = await r.json();
      document.getElementById('result').textContent = JSON.stringify(data, null, 2);
    }
  </script>
</body>
</html>`;

    res.writeHead(200, commonHeaders());
    return res.end(html);
  }

  res.writeHead(404, commonHeaders('application/json; charset=utf-8'));
  res.end(JSON.stringify({ error: 'Not found' }));
});

server.listen(PORT, '0.0.0.0', () => {
  console.log(`Distributed App Lab ${APP_VERSION} listening on port ${PORT}`);
  console.log(`Hostname: ${os.hostname()}`);
});

process.on('SIGTERM', () => {
  console.log('SIGTERM recibido. Cerrando servidor de forma controlada...');
  server.close(() => {
```

```
    console.log('Servidor cerrado.');
    process.exit(0);
  });

  setTimeout(() => process.exit(1), 10000).unref();
});
```

Aplicación v4 que delega el estado en el coordinador

```
const http = require('http');
const os = require('os');

const PORT = Number(process.env.PORT || 3000);
const COORDINATOR_URL =
    process.env.COORDINATOR_URL || 'http://counter-coordinator:3000';

const VERSION = 'v4';

function send(res, status, data) {
    if (res.writableEnded || res.destroyed) return;

    res.writeHead(status, {
        'Content-Type': 'application/json; charset=utf-8',
        'Cache-Control': 'no-store',
        'Connection': 'close',
    });

    res.end(JSON.stringify({
        ...data,
        pod: os.hostname(),
        version: VERSION,
    }));
}

function forward(req, res, path) {
    const headers = { Connection: 'close' };

    if (req.headers['idempotency-key']) {
        headers['Idempotency-Key'] = req.headers['idempotency-key'];
    }

    const upstream = http.request(
        new URL(path, COORDINATOR_URL),
        {
            method: req.method,
            headers,
            agent: false,
        },
        (response) => {
            let body = '';

            response.on('data', (chunk) => {
                body += chunk;
            });

            response.on('end', () => {
                try {
                    const data = JSON.parse(body);
                    send(res, response.statusCode, data);
                } catch (_) {
                    send(res, 502, {
                        error: 'Respuesta invalida del coordinador',
```

```
                    });
                }
            });

            response.on('error', () => {
                send(res, 502, {
                    error: 'Respuesta incompleta del coordinador',
                });
            });
        }
    );

    upstream.setTimeout(3000, () => {
        upstream.destroy(new Error('Timeout del coordinador'));
    });

    upstream.on('error', () => {
        send(res, 503, {
            error: 'No se pudo confirmar la operacion con el coordinador',
            instruction: 'Para reintentar un incremento, conserve Idempotency-Key',
        });
    });

    upstream.end();
}

const server = http.createServer((req, res) => {
    req.resume();
    const path = new URL(req.url, 'http://localhost').pathname;

    if (req.method === 'GET' && path === '/health') {
        return send(res, 200, { status: 'OK' });
    }

    if (
        req.method === 'GET' &&
        ['/', '/api/info', '/counter'].includes(path)
    ) {
        return forward(req, res, '/counter');
    }

    if (req.method === 'POST' && path === '/increment') {
        return forward(req, res, '/increment');
    }

    // Usar solamente para preparar pruebas sin trafico activo.
    if (req.method === 'POST' && path === '/reset') {
        return forward(req, res, '/reset');
    }

    return send(res, 404, { error: 'Ruta no encontrada' });
});

server.listen(PORT, '0.0.0.0', () => {
    console.log(`Aplicacion ${VERSION} en puerto ${PORT}`);
    console.log(`Pod: ${os.hostname()}`);
    console.log(`Coordinador: ${COORDINATOR_URL}`);
});

process.on('SIGTERM', () => {
    server.close(() => process.exit(0));
    setTimeout(() => process.exit(1), 10000).unref();
```

```
});
```

La aplicación pasó por variantes distintas. El server.js local v3 incluido aquí mantiene localCounter y fija APP_VERSION y APP_MESSAGE en el código; no lee esas dos variables del entorno. También se conserva completa la variante configurable de distributed-app-lab/server.js, que sí las lee y contiene la interfaz con pagos locales. Esto explica que una imagen etiquetada v2 pudiera responder v3 por su valor predeterminado hasta establecer APP_VERSION=v2. Una etiqueta Docker no determina el contenido de la respuesta. La aplicación v4 delega el contador en el coordinador. No se presenta el código v3 como una reconstrucción exacta de la versión inicial v1.

Dockerfile (imágenes base v1, v2 y v3)

```
FROM node:22-alpine
WORKDIR /app
COPY server.js .
EXPOSE 3000
CMD ["node", "server.js"]
```

Dockerfile (imagen v4 — reto de consistencia)

```
FROM node:22-alpine
WORKDIR /app
COPY server.js coordinator.js ./
EXPOSE 3000
CMD ["node", "server.js"]
```

El Dockerfile base copia únicamente server.js; el Dockerfile de v4 copia además coordinator.js, de modo que la misma imagen v4 sirve tanto para las réplicas (node server.js) como para el coordinador (node coordinator.js, definido en coordinator.yaml).

Manifiestos de Kubernetes

deployment.yaml (aplicación base)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: distributed-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: distributed-app
  template:
    metadata:
```

```
      labels:
        app: distributed-app
    spec:
      containers:
        - name: distributed-app
          image: distributed-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 3000
```

service.yaml (NodePort)

```
apiVersion: v1
kind: Service
metadata:
  name: distributed-app-service
spec:
  type: NodePort
  selector:
    app: distributed-app
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

application.yaml (réplicas v4)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: distributed-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app: distributed-app
  template:
    metadata:
      labels:
        app: distributed-app
    spec:
      terminationGracePeriodSeconds: 15
      containers:
        - name: distributed-app
          image: distributed-app:v4
          imagePullPolicy: Never
          command: ["node", "server.js"]
          env:
            - name: COORDINATOR_URL
              value: "http://counter-coordinator:3000"
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 1
```

```
            periodSeconds: 2
```

coordinator.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: counter-coordinator
  labels:
    app: counter-coordinator
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: counter-coordinator
  template:
    metadata:
      labels:
        app: counter-coordinator
    spec:
      terminationGracePeriodSeconds: 15
      containers:
        - name: coordinator
          image: distributed-app:v4
          imagePullPolicy: Never
          command: ["node", "coordinator.js"]
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 1
            periodSeconds: 2
---
apiVersion: v1
kind: Service
metadata:
  name: counter-coordinator
  labels:
    app: counter-coordinator
spec:
  type: ClusterIP
  selector:
    app: counter-coordinator
  ports:
    - port: 3000
      targetPort: 3000
```

deployment-stable.yaml (Canary: grupo estable v2)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: distributed-app-stable
spec:
  replicas: 4
  selector:
    matchLabels:
      app: distributed-app
```

```
      track: stable
  template:
    metadata:
      labels:
        app: distributed-app
        track: stable
    spec:
      containers:
        - name: distributed-app
          image: distributed-app:v2
          imagePullPolicy: Never
          env:
            - name: APP_VERSION
              value: "v2"
            - name: APP_MESSAGE
              value: "Versión ESTABLE (v2)"
          ports:
            - containerPort: 3000
```

deployment-canary.yaml (Canary: versión candidata v3)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: distributed-app-canary
spec:
  replicas: 1
  selector:
    matchLabels:
      app: distributed-app
      track: canary
  template:
    metadata:
      labels:
        app: distributed-app
        track: canary
    spec:
      containers:
        - name: distributed-app
          image: distributed-app:v3
          imagePullPolicy: Never
          env:
            - name: APP_VERSION
              value: "v3-canary"
            - name: APP_MESSAGE
              value: "Versión CANARY (v3) - En prueba"
          ports:
            - containerPort: 3000
```

En deployment-canary.yaml se configura APP_VERSION=v3-canary, pero la imagen v3 fija la versión en server.js y no lee esa variable; las capturas muestran v3. En el grupo estable, la variante configurable sí responde al valor APP_VERSION=v2. Para interpretar el reparto se usaron las versiones efectivamente devueltas, v2 y v3.

Es importante distinguir dos configuraciones. El deployment.yaml base (apartado «Desarrollo del laboratorio») declara replicas 3, imagePullPolicy Never y NO incluye readinessProbe ni un bloque strategy explícito, por lo que utiliza la estrategia RollingUpdate por defecto de Kubernetes (maxUnavailable y maxSurge del 25 %). En cambio, la configuración v4 (application.yaml y coordinator.yaml, apartado «Consistencia del estado distribuido») sí agrega la readinessProbe sobre /health y una estrategia explícita (RollingUpdate con maxUnavailable 0 y maxSurge 1 en las réplicas, y Recreate en el coordinador). El Service NodePort publica el puerto 80 del clúster hacia el 3000 del contenedor, accesible desde el host en el 30080.

Scripts de prueba con k6

smoke-test.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 1,
  duration: '10s',
};

export default function () {
  const res = http.get('http://localhost:30080/api/info');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response contains version': (r) => r.body.includes('version'),
  });
  if (res.status === 200) {
    const data = res.json();
    console.log(`VU ${__VU} | Iteration ${__ITER} | Pod: ${data.pod} | Version:
${data.version}`);
  }
  sleep(1);
}
```

baseline-test.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
    vus: 200,
    duration: '30s',

    thresholds: {
        http_req_failed: ['rate<0.01'],
        http_req_duration: ['p(95)<250'],
    },
```

```
};

export default function () {

    const res = http.get(
        'http://localhost:30080/api/info'
    );

    check(res, {
        'status is 200': (r) => r.status === 200,
    });

    if (res.status === 200) {

        const data = res.json();

        // Mostrar solo las primeras 2 iteraciones
        // de cada usuario virtual.
        if (__ITER < 2) {

            console.log(
                `VU ${__VU} | ` +
                `Iteration ${__ITER} | ` +
                `Pod: ${data.pod} | ` +
                `Version: ${data.version}`
            );
        }
    }

    sleep(0.1);
}
```

resilience-test.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
    stages: [
        { duration: '10s', target: 20 },
        { duration: '40s', target: 20 },
        { duration: '10s', target: 0 },
    ],

    thresholds: {
        http_req_failed: ['rate<0.01'],
        http_req_duration: ['p(95)<500'],
        checks: ['rate>0.99'],
    },
};

export default function () {

    const res = http.get(
        'http://localhost:30080/api/info'
    );

    const success = check(res, {
        'status is 200': (r) => r.status === 200,
    });

    if (success) {
```

```
        const data = res.json();

        // Mostrar una muestra para no saturar la consola
        if (__ITER % 10 === 0) {

            console.log(
                `VU ${__VU} | ` +
                `Iteration ${__ITER} | ` +
                `Pod: ${data.pod} | ` +
                `Version: ${data.version}`
            );
        }
    }

    sleep(0.25);
}
```

update-test.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 10,
  duration: '60s',

  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<500'],
    checks: ['rate>0.99'],
  },
};

export default function () {

  const res = http.get('http://localhost:30080/api/info');

  const success = check(res, {
    'status is 200': (r) => r.status === 200,
  });

  if (success) {
    const data = res.json();

    // Mostrar una muestra periódica para observar
    // el cambio de Pods y versiones.
    if (__ITER % 5 === 0) {
      console.log(
        `VU ${__VU} | ` +
        `Iteration ${__ITER} | ` +
        `Pod: ${data.pod} | ` +
        `Version: ${data.version}`
      );
    }
  } else {
    console.error(
      `ERROR | VU ${__VU} | ` +
      `Iteration ${__ITER} | ` +
      `Status: ${res.status}`
    );
  }
```

```
  sleep(0.5);
}
```

load-test.js (punto de degradación)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
    vus: Number(__ENV.VUS || 20),
    duration: '30s',
    noConnectionReuse: true,
    thresholds: {
        http_req_failed: ['rate<0.01'],
        http_req_duration: ['p(95)<250'],
        checks: ['rate>0.99'],
    },
};

export default function () {
    const res = http.get('http://localhost:30080/api/info', {
        timeout: '5s',
    });

    check(res, {
        'status es 200': (r) => r.status === 200,
    });

    if (__VU <= 3 && __ITER === 0 && res.status === 200) {
        const data = res.json();
        console.log(
            `VU ${__VU} | Iteracion ${__ITER} | ` +
            `Pod: ${data.pod} | Version: ${data.version}`
        );
    }

    sleep(0.1);
}
```

canary-test.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Counter, Rate } from 'k6/metrics';

const solicitudesV2 = new Counter('solicitudes_v2');
const solicitudesV3 = new Counter('solicitudes_v3');
const proporcionV3 = new Rate('proporcion_v3');

export const options = {
    vus: 10,
    duration: '30s',
    noConnectionReuse: true,
    thresholds: {
        http_req_failed: ['rate<0.01'],
        http_req_duration: ['p(95)<500'],
        checks: ['rate>0.99'],
    },
};

export default function () {
```

```
    const res = http.get('http://localhost:30080/api/info', {
        timeout: '5s',
    });

    let data = {};
    try {
        data = res.json();
    } catch (_) { }

    const versionValida =
        data.version === 'v2' || data.version === 'v3';

    check(res, {
        'status es 200': (r) => r.status === 200,
        'version reconocida': () => versionValida,
    });

    if (res.status === 200 && versionValida) {
        solicitudesV2.add(data.version === 'v2' ? 1 : 0);
        solicitudesV3.add(data.version === 'v3' ? 1 : 0);
        proporcionV3.add(data.version === 'v3');

        if (__ITER < 3) {
            console.log(
                `VU ${__VU} | Iteracion ${__ITER} | ` +
                `Pod: ${data.pod} | Version: ${data.version}`
            );
        }
    }

    sleep(0.2);
}
```

consistency-test.js

```
import http from 'k6/http';
import { check } from 'k6';
import { Counter } from 'k6/metrics';

const BASE = 'http://localhost:30080';
const confirmados = new Counter('incrementos_confirmados');

export const options = {
    scenarios: {
        concurrencia: {
            executor: 'per-vu-iterations',
            vus: 100,
            iterations: 1,
            maxDuration: '30s',
        },
    },
    noConnectionReuse: true,
    thresholds: {
        http_req_failed: ['rate==0'],
        checks: ['rate==1'],
        incrementos_confirmados: ['count==100'],
    },
};

function readJson(res) {
    try {
        return res.json();
```

```
    } catch (_) {
        return {};
    }
}

export function setup() {
    const res = http.post(`${BASE}/reset`, null, {
        timeout: '5s',
    });
    const data = readJson(res);

    if (res.status !== 200 || data.counter !== 0 || !data.epoch) {
        throw new Error('No se pudo preparar el contador en cero');
    }

    console.log(`INICIAL: contador=${data.counter} | epoch=${data.epoch}`);
    return { epoch: data.epoch };
}

export default function (initial) {
    const operationId = `op-${__VU}`;

    const res = http.post(`${BASE}/increment`, null, {
        headers: {
            'Idempotency-Key': operationId,
        },
        timeout: '5s',
    });

    const data = readJson(res);

    const ok = check(res, {
        'incremento devuelve HTTP 200': (r) => r.status === 200,
        'operacion nueva': () => data.repeated === false,
        'identificador correcto': () => data.operationId === operationId,
        'contador entre 1 y 100': () =>
            Number.isInteger(data.counter) &&
            data.counter >= 1 &&
            data.counter <= 100,
        'coordinador no reiniciado': () => data.epoch === initial.epoch,
    });

    if (ok) confirmados.add(1);

    if (__VU <= 10) {
        console.log(
            `VU ${__VU} | Iteracion ${__ITER} | ` +
            `Pod: ${data.pod} | Version: ${data.version} | ` +
            `Operacion: ${operationId} | Contador: ${data.counter}`
        );
    }
}

export function teardown(initial) {
    const res = http.get(`${BASE}/counter`, {
        timeout: '5s',
    });
    const data = readJson(res);

    check(res, {
        'consulta final HTTP 200': (r) => r.status === 200,
        'contador final exactamente 100': () => data.counter === 100,
```

```
        '100 operaciones registradas': () => data.operations === 100,
        'misma instancia del coordinador': () => data.epoch === initial.epoch,
    });

    console.log(`RESULTADO FINAL: ${JSON.stringify(data)}`);
}
```

En estos scripts, options define la carga: vus (usuarios virtuales), duration o stages (etapas de subida y bajada), sleep (pausa entre iteraciones), noConnectionReuse (una conexión nueva por solicitud) y thresholds (criterios de aprobación sobre http_req_failed, http_req_duration y checks). Los checks validan cada respuesta y las métricas personalizadas (solicitudes_v2/v3 e incrementos_confirmados) permiten medir el reparto Canary y la concurrencia. El load-test.js usa un timeout de 5 s por solicitud y registra solo la primera iteración de los tres primeros VUs. Conviene notar que el baseline-test.js vigente fija vus 200; las corridas históricas del baseline que aparecen en las Figuras 27 y 28 se ejecutaron con otro número de VUs (según se lee en cada captura), porque el archivo se modificó entre ejecuciones.

Coordinador del estado distribuido

El coordinador mantiene un único contador y un mapa de operaciones ya procesadas (processed) indexado por Idempotency-Key, lo que garantiza que reenviar la misma operación no vuelva a incrementar. La réplica v4 (server.js del apartado «Aplicación y contenedores») no guarda estado: reenvía cada /increment al coordinador mediante el Service interno counter-coordinator:3000 y responde 503 si el coordinador no está disponible.

coordinator.js (coordinador central del contador)

```
const http = require('http');
const os = require('os');
const { randomUUID } = require('crypto');

const PORT = Number(process.env.PORT || 3000);
const epoch = randomUUID();

let counter = 0;
const processed = new Map();

function send(res, status, data) {
    res.writeHead(status, {
        'Content-Type': 'application/json; charset=utf-8',
```

```
        'Cache-Control': 'no-store',
        'Connection': 'close',
    });
    res.end(JSON.stringify(data));
}

const server = http.createServer((req, res) => {
    req.resume();
    const path = new URL(req.url, 'http://localhost').pathname;

    if (req.method === 'GET' && path === '/health') {
        return send(res, 200, { status: 'OK', epoch });
    }

    if (req.method === 'GET' && path === '/counter') {
        return send(res, 200, {
            counter,
            operations: processed.size,
            coordinatorPod: os.hostname(),
            epoch,
        });
    }

    // Reinicio manual para preparar experimentos sin trafico activo.
    if (req.method === 'POST' && path === '/reset') {
        counter = 0;
        processed.clear();
        return send(res, 200, { counter, epoch });
    }

    if (req.method === 'POST' && path === '/increment') {
        const id = req.headers['idempotency-key'];

        if (typeof id !== 'string' || !id.trim() || id.length > 128) {
            return send(res, 400, {
                error: 'Se requiere Idempotency-Key de 1 a 128 caracteres',
            });
        }

        // Consulta e incremento sin esperas asincronas entre ambos.
        if (processed.has(id)) {
            return send(res, 200, {
                ...processed.get(id),
                repeated: true,
            });
        }

        counter += 1;

        const result = {
            counter,
            operationId: id,
            coordinatorPod: os.hostname(),
            epoch,
            repeated: false,
        };

        processed.set(id, result);
        return send(res, 200, result);
    }

    return send(res, 404, { error: 'Ruta no encontrada' });
```

```
});

server.listen(PORT, '0.0.0.0', () => {
    console.log(`Coordinador activo en puerto ${PORT}`);
    console.log(`Pod: ${os.hostname()} | Epoch: ${epoch}`);
});

process.on('SIGTERM', () => {
    server.close(() => process.exit(0));
    setTimeout(() => process.exit(1), 10000).unref();
});
```

Evidencias experimentales con k6

Parámetros y métricas de k6

Antes de presentar las pruebas conviene fijar el significado de cada parámetro y métrica, porque la calificación valora la interpretación y no solo el número:

- VUs (usuarios virtuales): número de clientes concurrentes que k6 simula; cada VU ejecuta el script en bucle.
- duration / stages: la duración fija la ventana de carga; las stages definen rampas de subida y bajada (por ejemplo 10 s hasta 20 VUs, 40 s en 20 y 10 s hasta 0).
- sleep: pausa entre iteraciones de cada VU (0,1 s o 0,25 s en estas pruebas); regula el ritmo de solicitudes.
- checks: validaciones por respuesta (p. ej. status 200); su tasa debe ser alta.
- thresholds: criterios de aprobación; aquí http_req_failed rate<0,01 (menos del 1 % de errores) y http_req_duration p(95)<250 ms o <500 ms.
- http_reqs y solicitudes/s (Requests/s): total de peticiones y su tasa por segundo; miden el rendimiento.
- http_req_failed: porcentaje y número de solicitudes fallidas.
- avg, p90, p95 y max: latencias: promedio, percentiles 90 y 95 (el p95 es el umbral de calidad) y el valor máximo observado.

Se distingue siempre la duración configurada de la observada: por ejemplo, la prueba de actualización se configuró a 60 s pero se observó 1 min 24 s porque un timeout de ~60 s extendió el cierre controlado (gracefulStop).

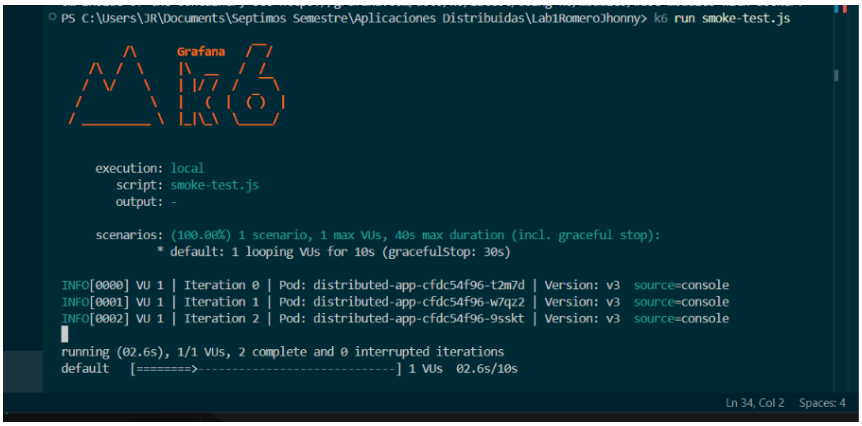
Smoke Test

Verificación mínima de funcionamiento: 1 VU durante 10 s contra /api/info, con checks de status 200 y presencia de version.

k6 arranca el escenario con 1 VU durante 10 s y registra en consola el Pod y la versión de cada iteración (todas v3). Sirve para confirmar, con carga mínima, que la aplicación responde correctamente antes de probar cargas mayores.

Figura 25

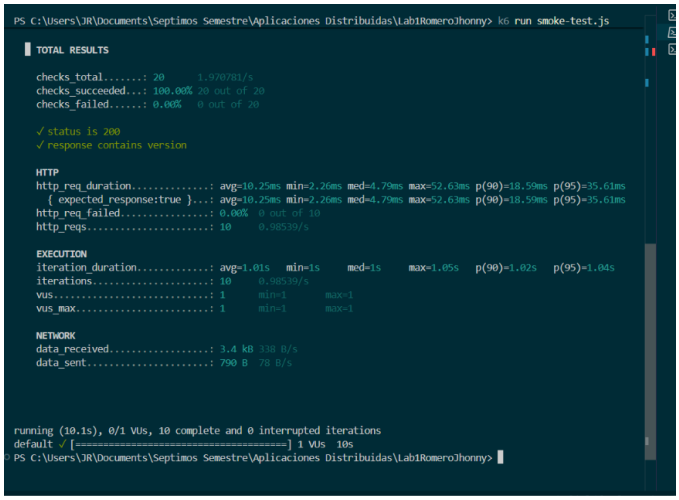
Inicio del Smoke Test (1 VU, 10 s)



La corrida completó 10 solicitudes (0,99/s) y 20 checks al 100 %, con http_req_failed 0,00 %, p(95) de 35,61 ms y máximo de 52,63 ms. El resultado PASS demuestra que las rutas y el Service funcionan correctamente bajo carga mínima.

Figura 26

Resultados del Smoke Test



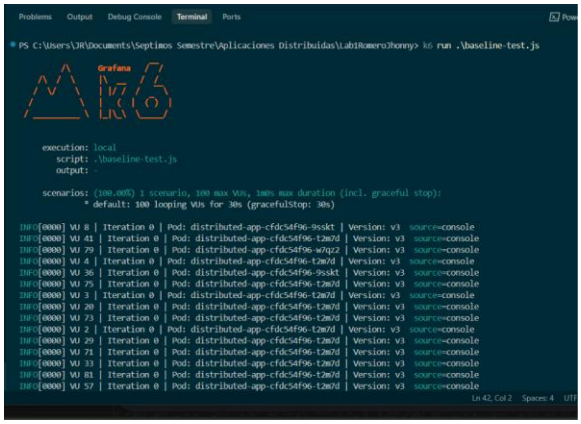
Baseline Test

El baseline busca una línea base de latencia (umbral p(95) < 250 ms). Dos aclaraciones importantes: la versión vigente de baseline-test.js fija vus 200, pero las corridas capturadas se ejecutaron con el número de VUs que indica cada captura, porque el archivo se editó entre ejecuciones; además, el inicio (Figura 27) y los resultados (Figura 28) corresponden a dos ejecuciones distintas, con distinto número de réplicas, por lo que no deben leerse como una sola corrida.

Esta captura muestra el inicio de una ejecución del baseline con 100 VUs durante 30 s, atendida por tres Pods (ReplicaSet de v3). Es solo el arranque de esa corrida; sus resultados no son los de la Figura 28, que corresponde a otra ejecución con una sola réplica.

Figura 27

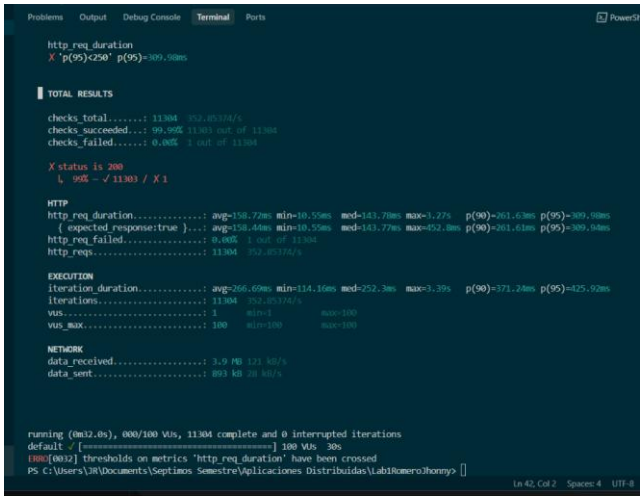
Inicio de una ejecución del Baseline con tres Pods



Esta es una ejecución distinta, realizada con una sola réplica (según se identificó): 11 304 solicitudes (352,85/s) con http_req_failed de 0,01 % (1 solicitud no-200 de 11 304), avg 158,72 ms, p(90) 261,63 ms y p(95) 309,98 ms, por lo que el umbral p(95) < 250 ms se incumplió (ERRO: threshold crossed). El máximo de 3,27 s indica al menos una solicitud muy lenta, pero ese único valor no explica por sí solo el p(95), que indica que aproximadamente el 95 % de las solicitudes tardó ese valor o menos. Sin mediciones de recursos no se atribuye la causa a un factor concreto; como posibles factores (hipótesis, no comprobados) estarían la única réplica frente a la carga, la contención en el nodo compartido o un calentamiento inicial. Por estas diferencias de condiciones, esta línea base no se compara directamente con la serie de degradación.

Figura 28

Resultados del Baseline con una réplica



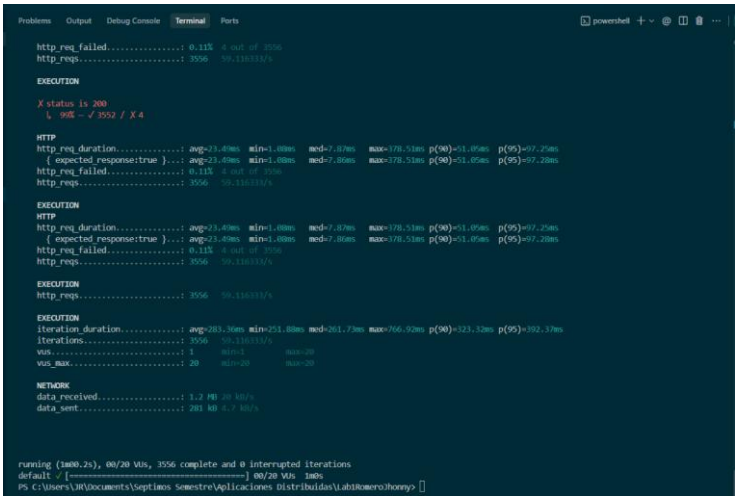
Resilience Test

Ejecución de carga (rampa 0→20→0 VUs durante 60 s) mientras se elimina un Pod, con umbrales p(95) < 500 ms, errores < 1 % y checks > 99 %.

Durante 60 s con hasta 20 VUs se completaron 3 556 solicitudes (59,12/s). Al eliminar un Pod bajo carga se produjeron 4 solicitudes fallidas (http_req_failed 0,11 %) y los checks quedaron en 99,89 %; el p(95) fue de 97,25 ms y el máximo de 378,51 ms. Como 0,11 % < 1 % y 99,89 % > 99 %, los umbrales se cumplieron: el sistema siguió disponible y se recuperó solo, pero las cuatro fallas evidencian que la caída de un Pod no es completamente transparente para el cliente.

Figura 29

Resultados del Resilience Test



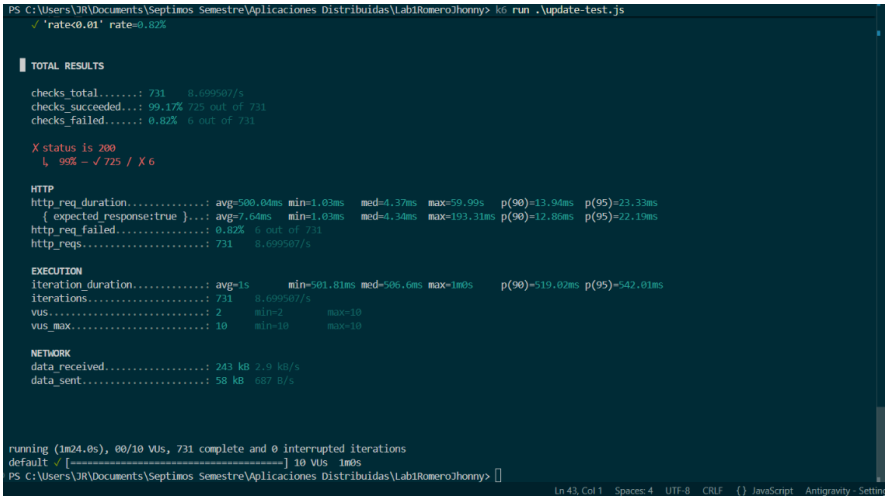
Update Test

Generación de tráfico (10 VUs, 60 s) durante un Rolling Update de v2 a v3.

La prueba completó 731 solicitudes (8,70/s) durante un Rolling Update, con 6 timeouts (http_req_failed 0,82 %, checks 99,17 %). Las métricas globales fueron: promedio 500,04 ms, p(90) 13,94 ms, p(95) 23,33 ms y máximo 59,99 s; los seis tiempos de espera registrados durante el cambio, cercanos a 60 s, elevan el promedio global; las capturas no identifican por sí solas la causa exacta de cada timeout. Medidas solo sobre las respuestas exitosas (claramente separadas), las latencias bajan a promedio 7,64 ms, p(95) 22,19 ms y máximo 193,31 ms. La duración observada (1 min 24 s) superó la configurada (60 s) por ese cierre prolongado. El umbral de errores se cumplió (0,82 % < 1 %), pero los seis timeouts muestran que la actualización en caliente no fue completamente transparente: hubo fallos transitorios durante el reemplazo de Pods.

Figura 30

Resultados del Update Test



Resultados de k6 y tabla comparativa

La Tabla 1 resume las cuatro pruebas con los valores reales transcritos de las capturas, distinguiendo la duración configurada de la observada.

Tabla 1

Resumen de las pruebas k6 (valores reales de las capturas)

Configuración y volumen de solicitudes

Prueba	VUs	Dur. cfg.	Dur. obs.	http_reqs	Req/s	Fallos
Smoke	1	10 s	10,1 s	10	0,99	0,00 %
Baseline (1 répl.)	100	30 s	32,0 s	11 304	352,85	0,01 % (1)
Resilience	0→20→0	60 s	60,2 s	3 556	59,12	0,11 % (4)
Update (global)	10	60 s	84,0 s	731	8,70	0,82 % (6)

Latencias y validaciones de las mismas corridas

Prueba	avg	p(90)	p(95)	max	Checks	Result.
Smoke	10,25 ms	18,59 ms	35,61 ms	52,63 ms	100 %	PASS
Baseline (1 répl.)	158,72 ms	261,63 ms	309,98 ms	3,27 s	99,99 %	p(95) falla
Resilience	23,49 ms	51,05 ms	97,25 ms	378,51 ms	99,89 %	PASS (4 fallos)
Update (global)	500,04 ms	13,94 ms	23,33 ms	59,99 s	99,17 %	PASS (6 timeouts)

Nota. en Update se reportan las métricas globales; medidas solo sobre las respuestas exitosas serían avg 7,64 ms, p(95) 22,19 ms y max 193,31 ms. El Baseline corresponde a una ejecución con una sola réplica a 100 VUs; la versión vigente de baseline-test.js usa 200 VUs (otra corrida posterior se incluye en el Apéndice B). Los experimentos de resiliencia y actualización se conservan con sus fallos (4 y 6, respectivamente): no se presentan como corridas sin errores.

Consistencia del estado distribuido

Alternativas consideradas

Como se vio en el apartado «Distribución de solicitudes y contadores locales independientes», cada réplica mantiene un contador local independiente, por lo que el mismo endpoint devuelve valores distintos. El reto consiste en que varias réplicas mantengan un único contador lógico sin base de datos externa. Se compararon tres alternativas:

Tabla 2

Comparación de alternativas de consistencia

Alternativa	Cómo funciona	Ventajas	Limitaciones
Coordinador central en memoria	Una sola instancia es la autoridad del contador; las réplicas le reenvían cada operación.	Consistencia fuerte simple; idempotencia centralizada; sin BD externa.	Punto único de fallo; sin persistencia ni alta disponibilidad del estado.
Líder y seguidores (replicación)	Un líder acepta escrituras y las replica a seguidores que sirven lecturas.	Tolera la caída de seguidores; lecturas escalables.	Mayor complejidad; posible lag de replicación (consistencia eventual en lecturas).
Archivo/almacenamiento compartido con control de concurrencia	Las réplicas leen y escriben un recurso común con bloqueo.	No requiere un proceso coordinador aparte.	El bloqueo serializa y añade latencia; el recurso compartido vuelve a ser un punto único.

Se eligió el coordinador central en memoria por ser la opción más simple que garantiza consistencia fuerte e idempotencia sin base de datos externa, adecuada para el alcance del laboratorio. Se asume de forma explícita su principal sacrificio: no ofrece persistencia ni alta disponibilidad del estado, como se demuestra más adelante.

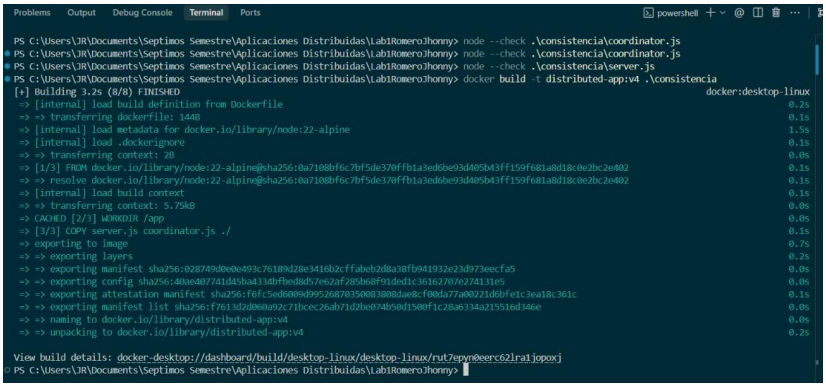
Implementación

Se construyó la imagen v4 con dos programas JavaScript: coordinator.js (una sola réplica, autoridad del contador y mapa de idempotencia) y server.js (tres réplicas que reenvían las operaciones al Service interno counter-coordinator:3000). El coordinador usa Recreate y readinessProbe sobre /health; el código completo figura en los apartados «Aplicación y contenedores» y «Coordinador del estado distribuido».

Se construyó distributed-app:v4 con docker build a partir de la carpeta consistencia, que contiene el coordinador y la réplica. Es la imagen común para el coordinador y las tres réplicas del experimento.

Figura 31

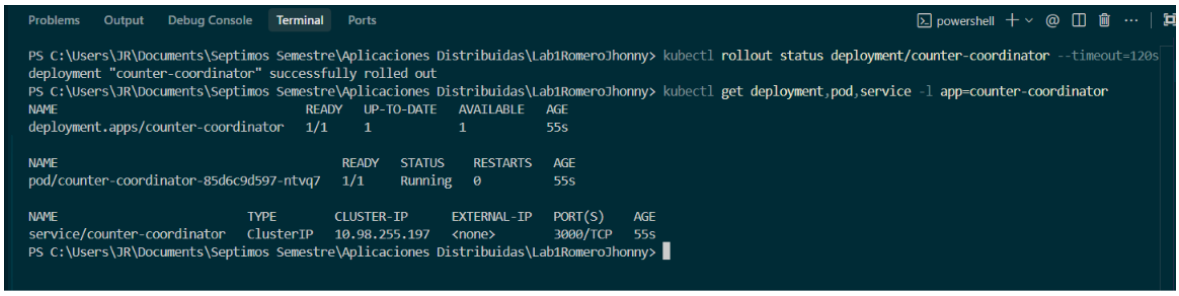
Construcción de la imagen v4 para la consistencia



El Deployment counter-coordinator queda en 1/1 y se expone con un Service ClusterIP counter-coordinator:3000 (interno, no accesible desde fuera). Las réplicas lo alcanzan por nombre dentro del clúster; al ser ClusterIP, el coordinador no se publica al exterior, solo el Service NodePort de la aplicación.

Figura 32

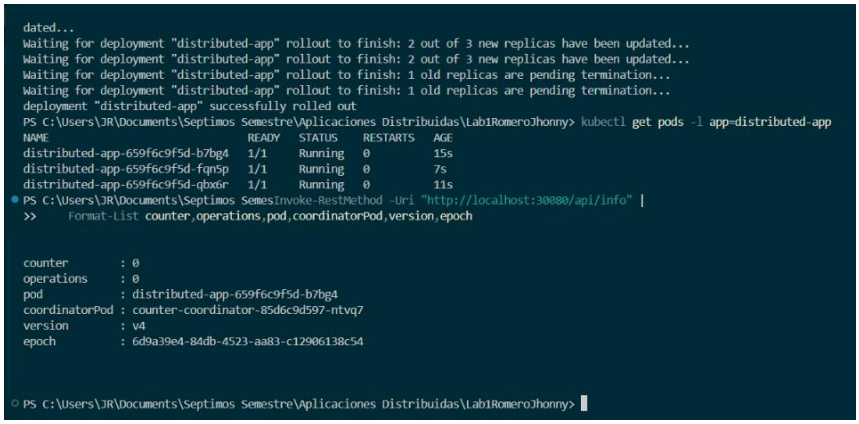
Coordinador central desplegado mediante Service interno



Tras el despliegue, las tres réplicas v4 están Running y /api/info devuelve counter 0, operations 0 y el epoch del coordinador. El epoch identifica la instancia viva del coordinador; servirá para detectar si este se reinicia.

Figura 33

Tres réplicas v4 conectadas al coordinador, estado inicial en cero



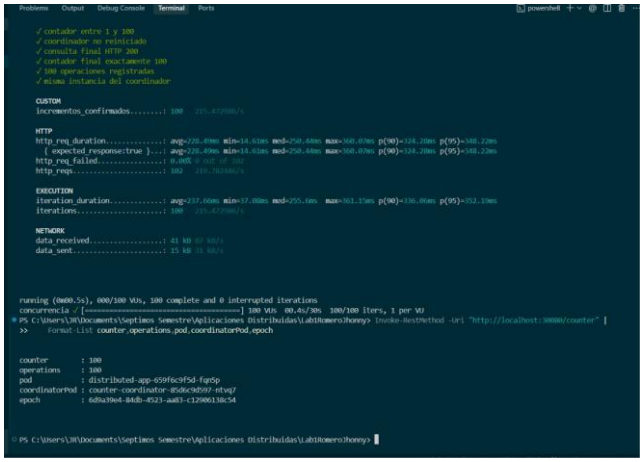
Prueba de concurrencia

Con consistency-test.js se lanzaron 100 VUs, cada uno con una iteración y una clave de idempotencia distinta (op-1 … op-100), tras reiniciar el contador a cero.

Los 100 incrementos se confirmaron, el contador final fue exactamente 100 y se registraron 100 operaciones, con cero errores HTTP; los checks pasaron al 100 %. El total de 102 solicitudes HTTP incluye el reinicio inicial (/reset) y la consulta final (/counter). Demuestra consistencia fuerte: 100 operaciones concurrentes produjeron el resultado lógico esperado sin pérdidas ni duplicados.

Figura 34

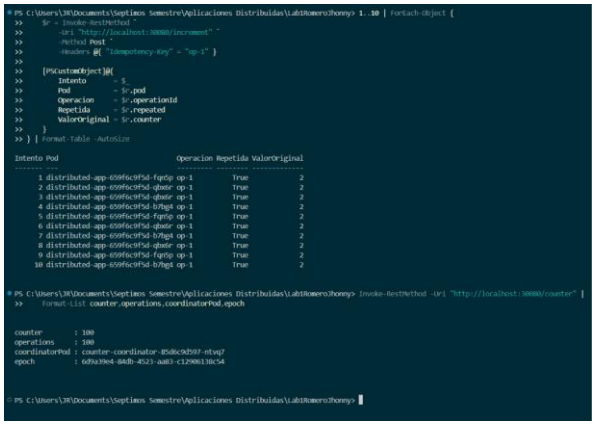
Prueba de 100 incrementos concurrentes



Al reenviar diez veces la misma operación op-1 —atendida por tres Pods distintos (fqn5p, qbx6r, b7bg4)— todas las respuestas llegan con repeated/Repetida = True y el mismo valor, y el contador global permanece en 100. Demuestra idempotencia coordinada: la deduplicación vive en el coordinador, no en la réplica que atiende.

Figura 35

Idempotencia: op-1 reenviada entre distintos Pods

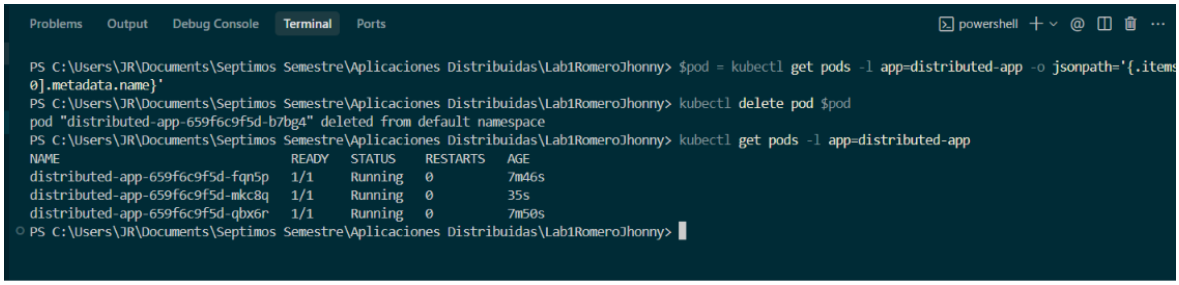


Reemplazo de una réplica y fallo del coordinador

Se eliminó un Pod de la aplicación y Kubernetes creó uno nuevo (mkc8q), quedando de nuevo tres Pods Running. Prepara la comprobación de si el estado sobrevive al cambio de una réplica.

Figura 36

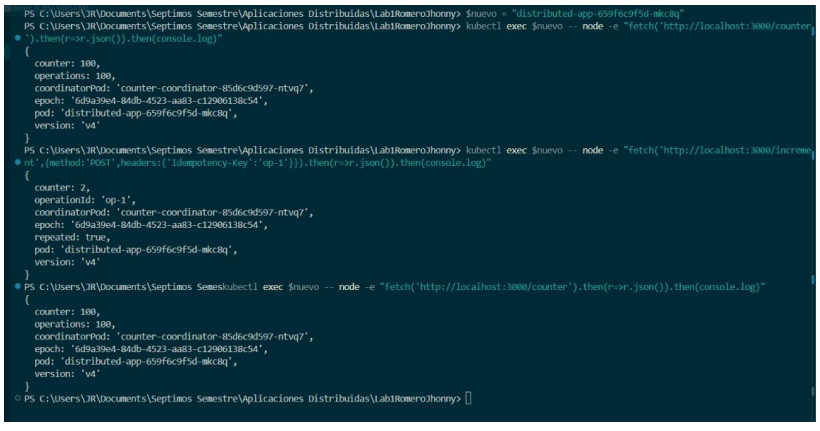
Reemplazo de una réplica durante el experimento



El Pod nuevo consultó el contador y obtuvo 100, y al reenviar op-1 la respuesta fue repeated: true sin volver a incrementar. Confirma que el estado no vive en las réplicas sino en el coordinador: una réplica nueva se incorpora y ve de inmediato el estado correcto.

Figura 37

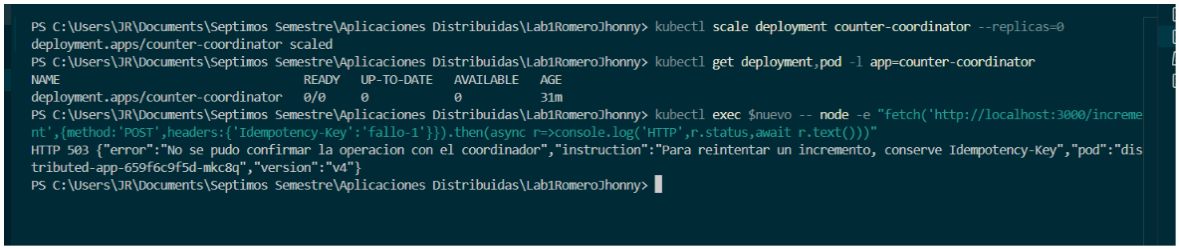
Estado e idempotencia conservados tras el reemplazo



Al escalar el coordinador a cero réplicas y luego intentar un incremento (clave fallo-1), la aplicación respondió HTTP 503 con el mensaje “No se pudo confirmar la operacion con el coordinador”. Demuestra la limitación esperada: sin coordinador no hay escritura posible; el sistema prioriza la consistencia sobre la disponibilidad del contador.

Figura 38

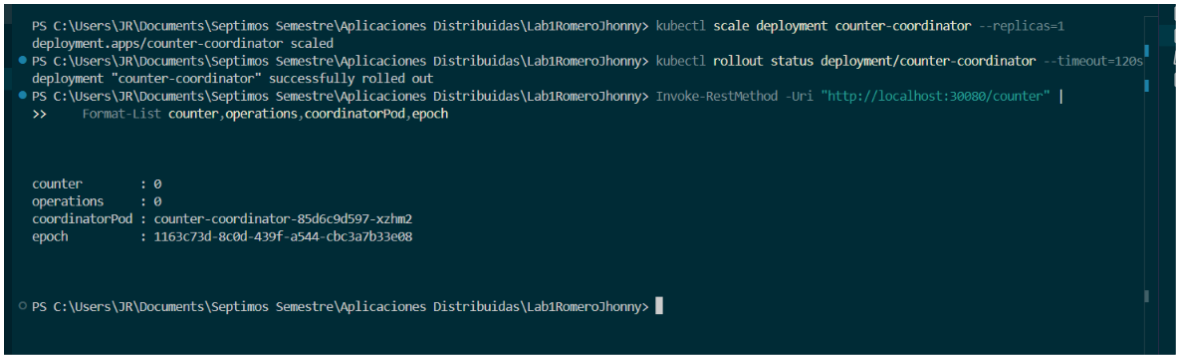
Fallo del coordinador: HTTP 503



Al volver a escalar el coordinador a una réplica, la consulta a /counter devuelve counter 0 y operations 0, y tanto el coordinatorPod (…-xzhm2) como el epoch son nuevos: el epoch cambió a un identificador distinto (1163c73d-…), no volvió a cero. El contador y el registro de operaciones sí se reinician a cero porque la instancia anterior y su estado en memoria desaparecieron. Demuestra que reiniciar el coordinador reinicia el estado: esta implementación no ofrece persistencia.

Figura 39

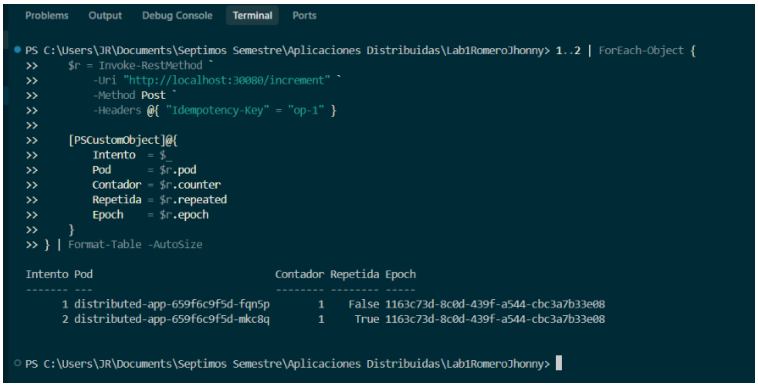
Reinicio del coordinador: contador y operaciones en cero con un nuevo epoch



Como el coordinador reinició (nuevo epoch), su mapa de operaciones también quedó vacío: al reenviar op-1, el primer intento la aceptó como nueva (Repetida False) y solo el segundo la reconoció como repetida (Repetida True). Es una evidencia distinta de la anterior: no solo se pierde el contador, también la deduplicación. La coordinación y la idempotencia funcionan únicamente durante la vida del coordinador: no hay alta disponibilidad del estado.

Figura 40

Pérdida del historial de idempotencia tras el reinicio



Análisis de concurrencia, fallos y escenarios hipotéticos

- Solicitudes simultáneas: 100 VUs incrementaron de forma concurrente y el contador final fue exacto (100), porque el coordinador serializa la consulta e incremento sin esperas asincrónicas entre ambos pasos.
- Pérdidas y duplicados: no hubo: cero errores HTTP y 100 operaciones registradas; cada clave op-N se contó una sola vez.
- Reintentos e idempotencia: reenviar op-1 entre distintos Pods no volvió a incrementar (repeated: true), lo que permite reintentar con seguridad conservando la Idempotency-Key. La clave debe conservarse ante un timeout: una respuesta perdida no permite saber si el incremento se aplicó. No se implementaron reintentos automáticos; los reenvíos mostrados fueron manuales. Tras un cambio de epoch, el historial anterior ya no permite deduplicar.
- Caída y reemplazo de una réplica: un Pod nuevo se incorporó y vio el estado 100 de inmediato, porque el estado no reside en la réplica.
- Recuperación del estado: al reiniciar el coordinador, counter y operations volvieron a cero, mientras epoch cambió a un nuevo UUID. También se perdió el mapa de deduplicación: op-1 fue aceptada de nuevo y después reconocida como repetida. Kubernetes recuperó el proceso, pero no los datos en memoria.
- Split-brain (hipotético): la estrategia Recreate termina el coordinador anterior antes de crear el nuevo durante una actualización normal, lo que reduce la probabilidad de dos autoridades a la vez, pero no garantiza por sí sola la ausencia de split-brain ante cualquier fallo (por ejemplo, una partición de red en la que una instancia antigua siga viva). Si

además se usaran varios coordinadores sin un mecanismo de consenso, sus contadores podrían divergir. No se probó.

- Partición de red (hipotético): no se realizó una partición real; si la red separara a las réplicas del coordinador, estas responderían 503 y no habría escritura, manteniendo la consistencia a costa de la disponibilidad.

En conclusión, esta solución ofrece consistencia fuerte e idempotencia mientras el coordinador está activo, pero sacrifica la disponibilidad y la durabilidad del estado cuando este falla. No se afirman garantías generales a partir de una prueba de 100 operaciones: para producción se requeriría persistencia y replicación con consenso (p. ej. líder-seguidores con quórum).

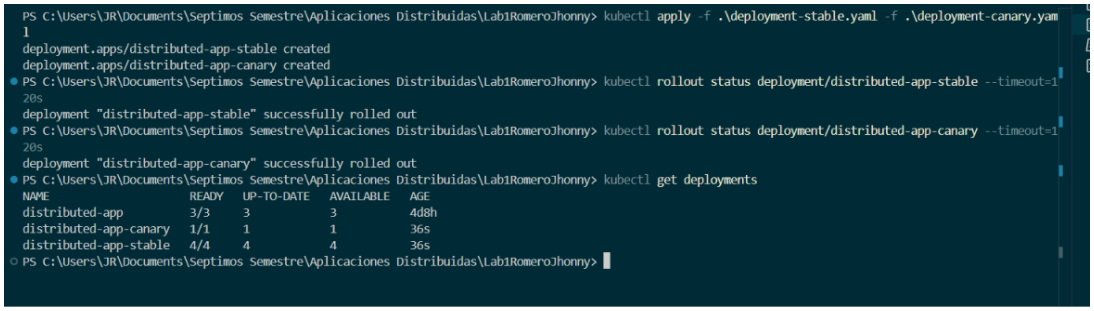
Canary Deployment

Se implementó una estrategia Canary con dos Deployments que comparten el selector app=distributed-app: un grupo estable con cuatro réplicas v2 y un grupo Canary con una réplica v3; el Deployment original se escaló a cero. Así, el Service distribuye el tráfico entre los cinco Pods, con una proporción teórica de 80 % a v2 y 20 % a v3.

Se aplicaron deployment-stable.yaml (4 réplicas v2) y deployment-canary.yaml (1 réplica v3); ambos quedaron disponibles (4/4 y 1/1). Es la base del reparto 80/20: cuatro de cinco Pods ejecutan la versión estable.

Figura 41

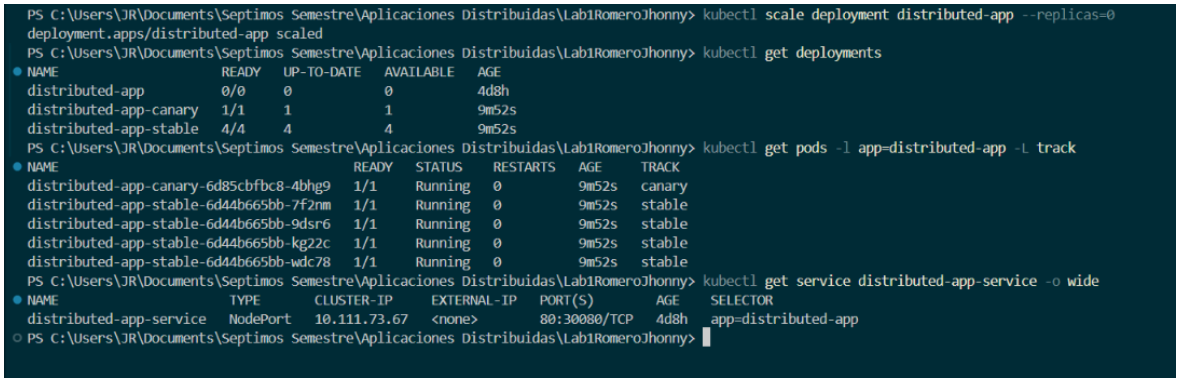
Grupos estable y Canary creados (4 + 1 réplicas)



Se escaló el Deployment original a 0 y se listaron los Pods con la etiqueta track (stable/canary); el Service distributed-app-service es NodePort (80→30080). Al compartir el selector, el Service reparte el tráfico entre los cuatro Pods stable y el Pod canary.

Figura 42

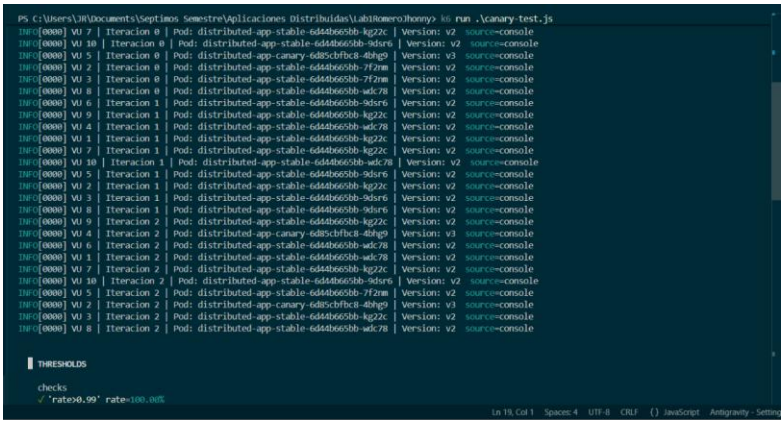
Deployment original en cero, etiquetas track y Service NodePort



El registro de k6 muestra, iteración a iteración, qué Pod y qué versión atendió cada solicitud: la mayoría de respuestas son v2 (Pods -stable-) y algunas v3 (Pod -canary-). Permite observar desde el cliente que ambas versiones reciben tráfico simultáneamente.

Figura 43

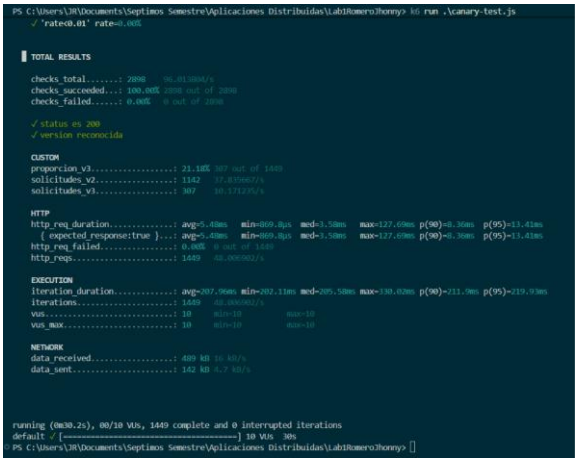
Ejecución del Canary Test



De 1 449 solicitudes, 1 142 fueron atendidas por v2 y 307 por v3, es decir 78,8 % y 21,2 % (la captura de k6 muestra proporcion_v3 = 21,18 %; el cociente 307/1 449 equivale aproximadamente a 21,19 %), con cero errores (http_req_failed 0,00 %), p(90) de 8,36 ms y p(95) de 13,41 ms. Confirma un reparto cercano al 80/20 esperado y un buen desempeño general. Importante: ese p(95) de 13,41 ms corresponde al tráfico agregado (v2 y v3 juntas); con este script no se midió por separado la latencia de cada versión, que es justamente lo que haría falta para comparar v3 frente a v2 antes de promoverla.

Figura 44

Resultados del Canary Test (reparto ≈ 78,8 % / 21,2 %)



Respuestas requeridas

- ¿Cómo implementó la distribución? con dos Deployments (stable v2 ×4 y canary v3 ×1) bajo el mismo selector; el Service NodePort reparte el tráfico entre los cinco Pods y el Deployment original se escaló a cero.
- ¿Qué porcentaje esperaba? aproximadamente 80 % v2 / 20 % v3 (cuatro de cada cinco Pods son estables).
- ¿Qué porcentaje observó? 78,8 % v2 (1 142) y 21,2 % v3 (307) de 1 449 solicitudes.
- ¿Por qué no es exactamente 80/20? el Service balancea por conexión de forma aproximada, no proporcional exacta; con noConnectionReuse cada solicitud puede ir a cualquier Pod y el reparto fluctúa por azar y por pequeñas diferencias de latencia entre Pods.
- ¿Qué métricas permitirían promover v3? la tasa de error y la latencia (p(95)) de v3 medidas por separado y comparadas con las de v2; deberían ser iguales o mejores, con checks al 100 %, sostenidas al aumentar su fracción de tráfico. En esta prueba el p(95) de

13,41 ms es agregado (no se separó por versión), por lo que para una decisión real habría que etiquetar las métricas por versión.

- ¿Qué condición justificaría un rollback? un aumento de errores o de latencia en v3 respecto a v2, o fallos funcionales; se revertiría escalando el Canary a cero y manteniendo el grupo estable.
- ¿Diferencia entre Canary y Rolling Update? el Rolling Update reemplaza todos los Pods gradualmente hasta dejar solo la versión nueva (la coexistencia es transitoria y no se controla por porcentaje); el Canary mantiene ambas versiones en paralelo y dirige solo una fracción del tráfico a la candidata para evaluarla con datos antes de promoverla.

Distinción entre lo propuesto y lo ejecutado: en esta práctica se ejecutó el reparto Canary y se midió su desempeño; los criterios de promoción y de rollback son decisiones propuestas: no se promovió v3 ni se ejecutó un rollback del Canary.

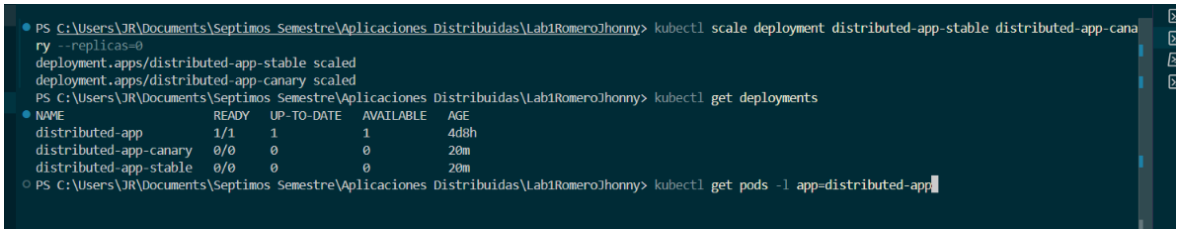
Punto de degradación del sistema

Se ejecutó el mismo script (load-test.js) con 20, 50, 100 y 200 VUs sobre 1, 3 y 10 réplicas, manteniendo constantes la duración (30 s), la pausa (sleep 0,1 s) y los umbrales (p(95) < 250 ms, errores < 1 %). Variar solo el número de réplicas y los VUs permite una comparación válida entre escenarios.

Se escaló el Deployment a una réplica (1/1) para la primera serie. Todas las solicitudes de ese escenario recaen en una instancia; esta configuración sirve como referencia para compararla con tres y diez réplicas.

Figura 45

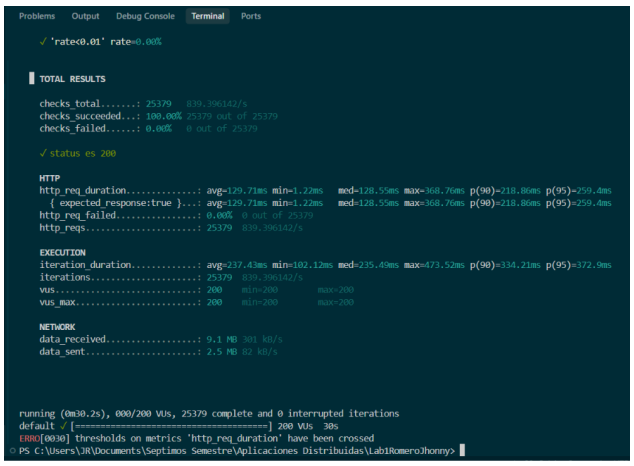
Escenario con una réplica



Con una réplica y 200 VUs se atendieron 25 379 solicitudes (839,40/s) sin errores, pero p(95) subió a 259,40 ms y cruzó 250 ms. Es el primer incumplimiento de latencia entre los niveles ensayados con una réplica, sin afirmar una capacidad máxima exacta.

Figura 46

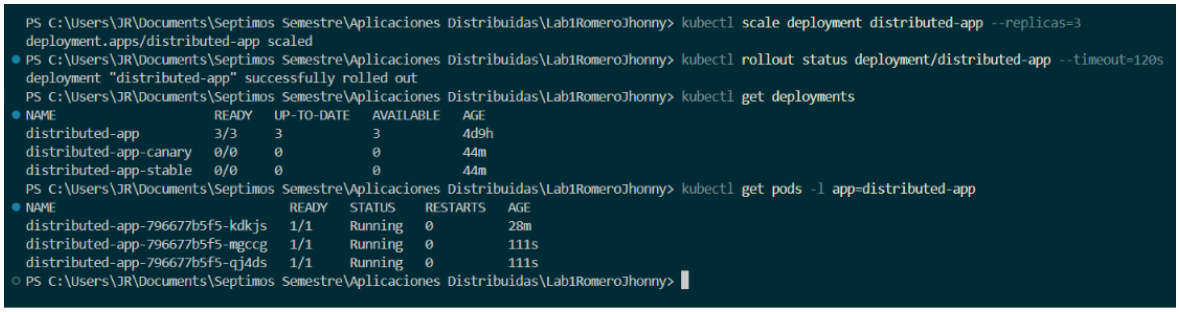
Una réplica, 200 VUs: primer incumplimiento del umbral



Se escaló el Deployment a tres réplicas (3/3). Con tres Pods la carga se reparte, aunque todos siguen compartiendo el mismo nodo docker-desktop.

Figura 47

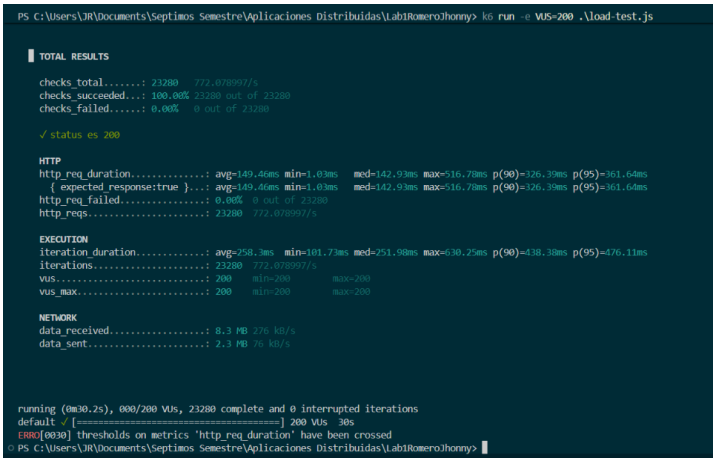
Escenario con tres réplicas



Con 3 réplicas y 200 VUs se atendieron 23 280 solicitudes (772,08/s) sin errores, con p(95) de 361,64 ms, también por encima de 250 ms. Observación: este p(95) es peor que con una réplica. Una posible explicación (hipótesis, no medida) es que, al compartir un solo nodo, más Pods compiten por los mismos recursos y añaden sobrecarga; confirmarlo requeriría medir CPU y memoria durante la prueba.

Figura 48

Tres réplicas, 200 VUs: incumplimiento del umbral

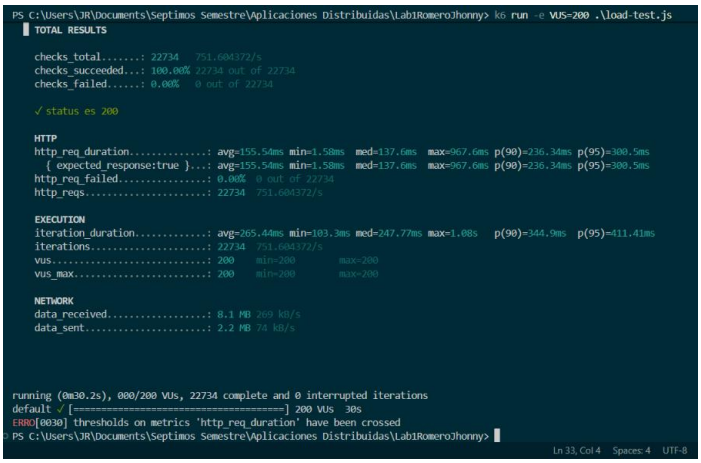


Con 10 réplicas y 200 VUs se atendieron 22 734 solicitudes (751,60/s) sin errores, con p(95) de 300,50 ms, igualmente por encima de 250 ms. Observación: diez Pods tampoco bajaron

el p(95) de 250 ms a 200 VUs. Esto es coherente con que los diez Pods comparten un único nodo (Figura 17), pero no se puede afirmar la causa sin medir los recursos del equipo.

Figura 49

Diez réplicas, 200 VUs: incumplimiento del umbral



La Tabla 3 reúne los doce escenarios con los valores reales de las capturas (las corridas de 20, 50 y 100 VUs se incluyen en el Apéndice A).

Tabla 3

Punto de degradación: 1, 3 y 10 réplicas × 20, 50, 100 y 200 VUs

Répl.	VUs	Solicitudes	Req/s	avg ms	p90 ms	p95 ms	max ms	Estado
1	20	5 662	188,07	4,30	7,35	11,39	61,56	PASS
1	50	13 567	450,57	8,23	16,91	22,83	223,54	PASS
1	100	19 652	652,49	48,35	108,96	155,72	402,91	PASS
1	200	25 379	839,40	129,71	218,86	259,40	368,76	FAIL
3	20	5 680	188,64	4,14	6,94	10,01	57,37	PASS
3	50	12 975	430,54	13,38	27,88	48,14	226,27	PASS
3	100	21 307	705,09	37,96	81,35	95,97	265,09	PASS
3	200	23 280	772,08	149,46	326,39	361,64	516,78	FAIL
10	20	5 369	178,36	8,42	17,03	30,86	182,15	PASS
10	50	12 958	430,44	13,55	27,99	38,34	288,52	PASS
10	100	18 930	627,92	53,59	101,14	114,47	267,88	PASS
10	200	22 734	751,60	155,54	236,34	300,50	967,60	FAIL

Interpretación del punto de degradación

El primer nivel de carga que incumplió el umbral p(95) < 250 ms fue 200 VUs en los tres escenarios (259,40 / 361,64 / 300,50 ms); a 100 VUs todos cumplían. Esto identifica el primer incumplimiento entre los niveles ensayados (20, 50, 100 y 200 VUs), no una capacidad máxima exacta: harían falta niveles intermedios y repeticiones para acotar mejor el inicio del incumplimiento. En todos los casos los errores se mantuvieron en 0 %; lo que se degradó fue la latencia, no la tasa de éxito. Se detuvo la progresión en 200 VUs porque ya se había observado el criterio de degradación; no se ejecutó 500 VUs.

No se atribuye la degradación a un cuello de botella específico sin mediciones que lo demuestren. Como posibles factores limitantes (hipótesis a verificar) se consideran: la CPU del

equipo, la memoria disponible, el propio Docker Desktop y su capa de virtualización, la red interna de Kubernetes y el generador k6, que también consume recursos en la misma máquina. El dato observado (Figura 17) es que los diez Pods se ejecutaban en el único nodo docker-desktop; esto es coherente con que agregar réplicas no mejorara la latencia a 200 VUs, pero confirmarlo exigiría medir el consumo de recursos durante cada corrida. Lo que sí se puede afirmar es que diez Pods en un nodo no equivalen a diez servidores físicos: comparten CPU, memoria y red del mismo equipo.

Análisis comparativo final

Tabla 4

Matriz comparativa de todos los experimentos

Experimento	Réplicas	VUs	Fallos %	p(95)	Resultado principal
Smoke Test	3	1	0,00 %	35,61 ms	Funcionamiento básico verificado
Baseline	1	100	0,01 %	309,98 ms	Línea base (1 réplica); p(95) cruzó el umbral de 250 ms
Resilience	3	0→20	0,11 %	97,25 ms	4 fallos durante la caída; recuperación automática
Rolling Update	3	10	0,82 %	23,33 ms	Actualización v2→v3 con 6 timeouts transitorios
Canary	4+1	10	0,00 %	13,41 ms	78,8 % v2 / 21,2 % v3; cero errores
Consistencia	3	100	0,00 %	348,22 ms	100/100 incrementos; idempotencia correcta
Degradación	1·3·10	200	0,00 %	259–362 ms	Primer incumplimiento de p(95)<250 a 200 VUs

Respuestas al análisis

- 1. ¿Qué escenario produjo la mayor degradación? 200 VUs con 3 réplicas, con el peor p(95) (361,64 ms).

- 2. ¿Agregar réplicas mejoró siempre el rendimiento? No. A 200 VUs, tres réplicas (361,64 ms) fueron peores que una (259,40 ms) y diez (300,50 ms); una explicación posible —no medida— es que, al compartir un único nodo, más Pods compiten por los mismos recursos.
- 3. ¿Diferencia entre disponibilidad y consistencia? disponibilidad es seguir respondiendo (self-healing, rolling update); consistencia es que todas las réplicas vean el mismo estado. El contador local fue disponible pero inconsistente; el coordinador fue consistente pero dejó de estar disponible al caer.
- 4. ¿Qué evidencia demuestra mejor que la app siguió disponible? el Resilience Test: 3 556 solicitudes con solo 4 fallos (0,11 %) mientras se eliminaba un Pod, y el Rolling Update, que mantuvo el servicio durante el cambio de versión pese a 6 timeouts transitorios.
- 5. ¿Qué comportamiento observó al eliminar un Pod? recreación automática; bajo carga, cuatro fallos transitorios; el Pod nuevo nace con el contador local en cero (o, con coordinador, ve el estado correcto).
- 6. ¿Qué ventaja tuvo el Rolling Update? Permitió sustituir v2 por v3 gradualmente y mantener respuestas durante el cambio. De 731 solicitudes, 725 tuvieron éxito y seis agotaron el tiempo de espera. El despliegue cumplió los umbrales configurados, aunque no hubo disponibilidad del 100 % para las solicitudes.
- 7. ¿Qué ventaja adicional ofrece Canary? evaluar la versión candidata con una fracción del tráfico (21 %) y métricas reales antes de promoverla, con rollback sencillo.

- 8. ¿Qué solución de consistencia usó y qué sacrificó? coordinador central en memoria; ganó consistencia fuerte e idempotencia y sacrificó persistencia y alta disponibilidad del estado.
- 9. ¿Cuál fue el punto de degradación? el primer incumplimiento de p(95)<250 ms apareció a 200 VUs en los tres escenarios; no es una capacidad máxima exacta.
- 10. ¿Qué cambiaría en una infraestructura real con varios nodos físicos? Distribuir las réplicas entre nodos aportaría CPU y memoria adicionales y podría mejorar la capacidad si el cuello de botella es paralelizable. La mejora no está garantizada: también influyen la red, las dependencias compartidas y el generador de carga. Para conservar el estado se necesitarían almacenamiento duradero y una estrategia de replicación y consenso.

  Repositorio de Git

  joel1251/P1Lab1RomeroJhonnyKubernetes

Conclusiones

- Kubernetes recreó los Pods eliminados y permitió actualizar de v2 a v3 manteniendo réplicas disponibles. Las pruebas registraron cuatro solicitudes fallidas de 3 556 en resiliencia y seis timeouts de 731 en la actualización. Ambas cumplieron sus umbrales, pero estos resultados descartan afirmar que el cambio fue transparente para todos los clientes.
- El estado en memoria no se replica ni persiste por sí solo: cada réplica mantuvo un contador local independiente, y un Pod recreado nació en cero. La disponibilidad del proceso no implica la preservación del estado.

- El coordinador central produjo el resultado esperado de consistencia en su periodo de funcionamiento: 100 incrementos concurrentes, contador final 100 y cero errores. Reenviar op-1 no incrementó otra vez mientras se conservó el mismo epoch. Al detenerlo hubo HTTP 503; al recrearlo se perdieron el contador y las claves, apareció otro epoch y op-1 se aceptó como nueva. La solución carece de durabilidad y alta disponibilidad del estado.
- El Canary Deployment repartió el tráfico 78,8 % v2 / 21,2 % v3 con cero errores y p(95) de 13,41 ms, mostrando cómo evaluar una versión candidata con una fracción del tráfico antes de promoverla, con más control que el Rolling Update.
- El primer incumplimiento observado de p(95) < 250 ms ocurrió a 200 VUs: 259,40 ms con una réplica, 361,64 ms con tres y 300,50 ms con diez. Agregar Pods en el mismo nodo no mejoró esa latencia. No se midieron recursos para identificar la causa. Varios nodos podrían aportar capacidad adicional, pero su efecto tendría que comprobarse con la misma prueba y mediciones de recursos.

Referencias

Docker, Inc. (s. f.-a). *Dockerfile reference*. Recuperado el 19 de octubre de 2026, de https://docs.docker.com/reference/dockerfile/

Docker, Inc. (s. f.-b). *Kubernetes on Docker Desktop*. Recuperado el 19 de octubre de 2026, de https://docs.docker.com/desktop/kubernetes/

Grafana Labs. (s. f.-a). *Metrics*. Recuperado el 19 de octubre de 2026, de https://grafana.com/docs/k6/latest/using-k6/metrics/

Grafana Labs. (s. f.-b). *Thresholds*. Recuperado el 19 de octubre de 2026, de https://grafana.com/docs/k6/latest/using-k6/thresholds/

Node.js. (s. f.). *HTTP — Node.js v22*. Recuperado el 19 de octubre de 2026, de https://nodejs.org/docs/latest-v22.x/api/http.html

The Kubernetes Authors. (s. f.-a). *Configure liveness, readiness and startup probes*. Recuperado el 19 de octubre de 2026, de https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

The Kubernetes Authors. (s. f.-b). *Deployments*. Recuperado el 19 de octubre de 2026, de https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

The Kubernetes Authors. (s. f.-c). *Service*. Recuperado el 19 de octubre de 2026, de https://kubernetes.io/docs/concepts/services-networking/service/

Apéndice A

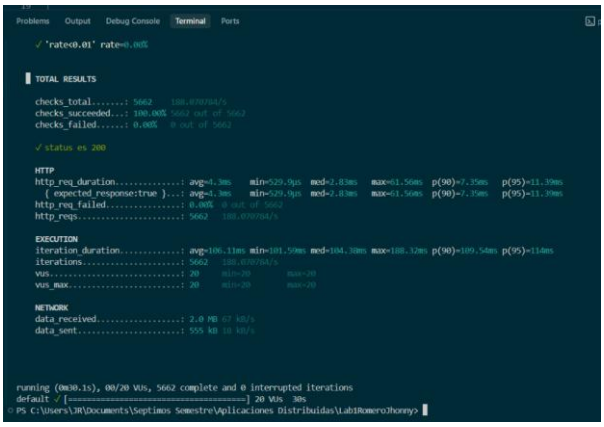
Corridas de carga adicionales con 20, 50 y 100 VUs

Se incluyen aquí las corridas del punto de degradación que cumplieron el umbral (20, 50 y 100 VUs en 1, 3 y 10 réplicas), cuyos valores ya figuran en la Tabla 3. Se conservan como evidencia completa.

188,07 solicitudes/s, 0 % de errores y p(95) de 11,39 ms: cumple holgadamente.

Figura 50

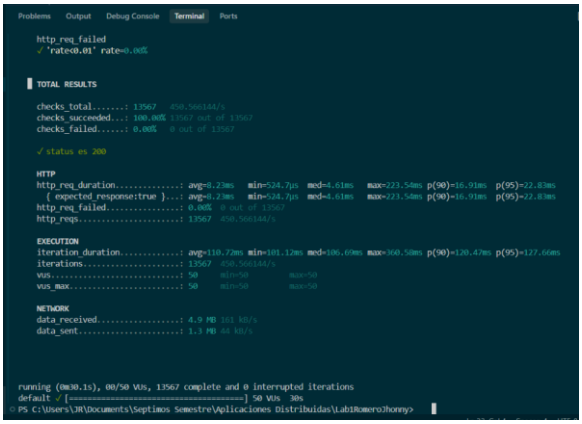
Una réplica, 20 VUs



450,57 solicitudes/s, 0 % de errores y p(95) de 22,83 ms: cumple.

Figura 51

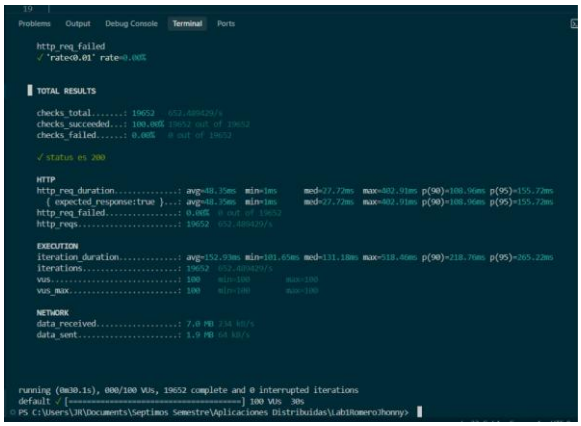
Una réplica, 50 VUs



652,49 solicitudes/s, 0 % de errores y p(95) de 155,72 ms: cumple, con latencia ya en aumento.

Figura 52

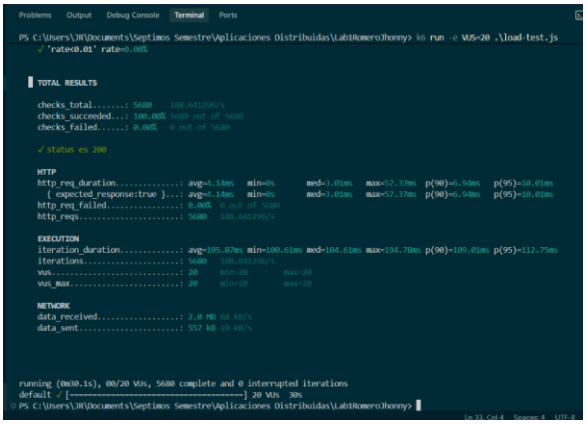
Una réplica, 100 VUs



188,64 solicitudes/s, 0 % de errores y p(95) de 10,01 ms: cumple.

Figura 53

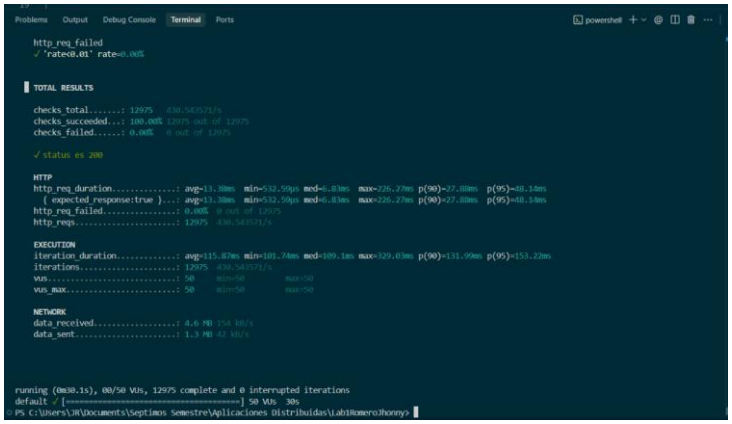
Tres réplicas, 20 VUs



430,54 solicitudes/s, 0 % de errores y p(95) de 48,14 ms: cumple.

Figura 54

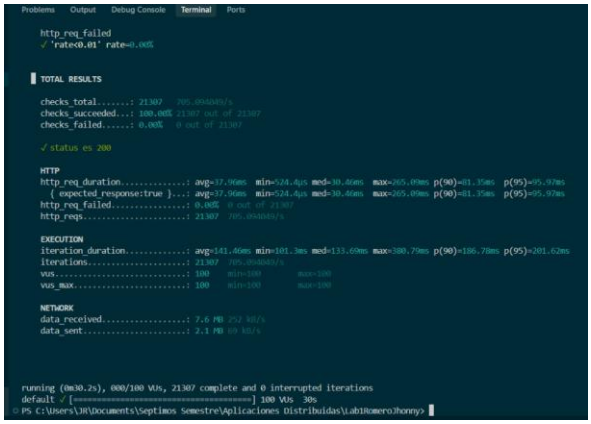
Tres réplicas, 50 VUs



705,09 solicitudes/s, 0 % de errores y p(95) de 95,97 ms: cumple.

Figura 55

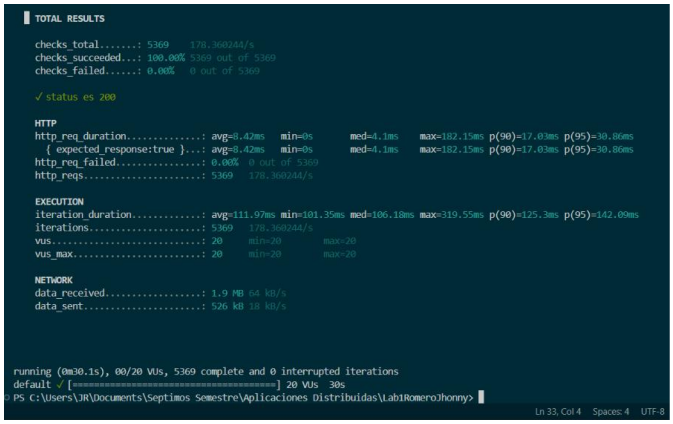
Tres réplicas, 100 VUs



178,36 solicitudes/s, 0 % de errores y p(95) de 30,86 ms: cumple.

Figura 56

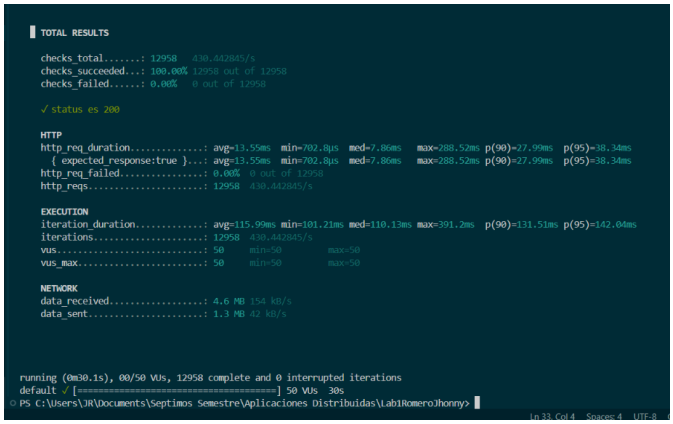
Diez réplicas, 20 VUs



430,44 solicitudes/s, 0 % de errores y p(95) de 38,34 ms: cumple.

Figura 57

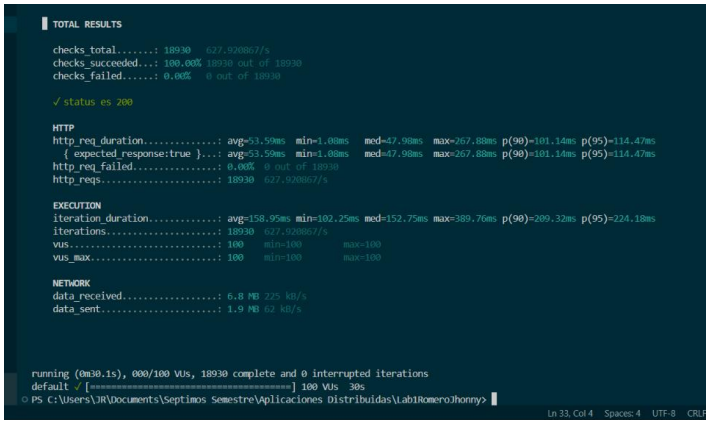
Diez réplicas, 50 VUs



627,92 solicitudes/s, 0 % de errores y p(95) de 114,47 ms: cumple.

Figura 58

Diez réplicas, 100 VUs



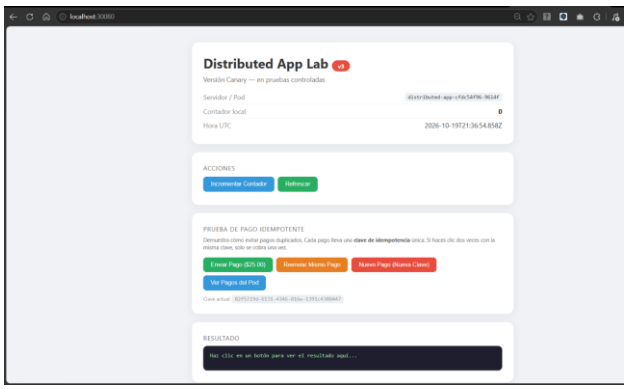
Apéndice B

Evidencias complementarias

La interfaz v3 muestra versión, Pod, contador local, hora UTC y una clave de idempotencia; se usó para pruebas manuales de incremento y reenvío.

Figura 59

Interfaz de la aplicación v3 con panel de idempotencia



Script mínimo de k6 (1 VU, 10 s) con checks de status 200 y presencia de version; su resultado se presenta en la Figura 26.

Figura 60

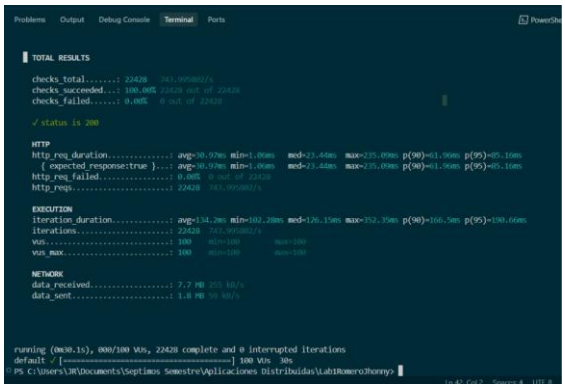
Código fuente de smoke-test.js



Corrida adicional con 100 VUs que atendió 22 428 solicitudes con 0 % de errores y p(95) de 85,16 ms, sin incumplir umbrales; se conserva como referencia de carga.

Figura 61

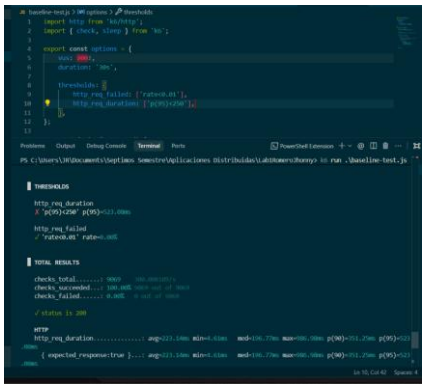
Corrida de carga de referencia (100 VUs, 30 s)



Muestra las opciones del baseline (vus 200, thresholds p(95)<250) y una corrida posterior cuyo p(95) de 523,08 ms también cruzó el umbral; evidencia que el archivo se ejecutó con 200 VUs en una corrida distinta de la de la Figura 28.

Figura 62

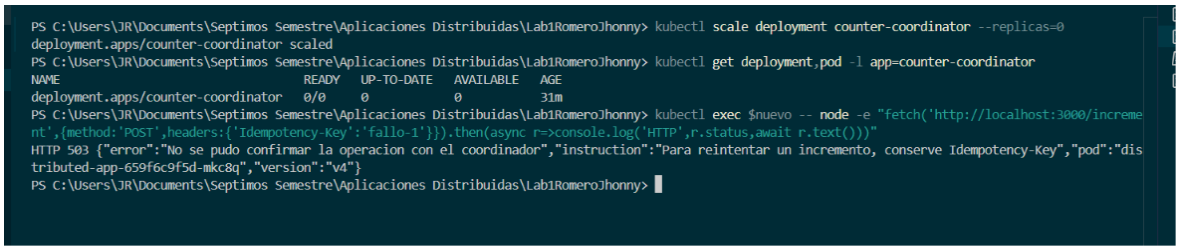
Fuente de baseline-test.js y segunda corrida



Captura adicional del fallo del coordinador: se escala el Deployment counter-coordinator a 0 réplicas (0/0) y un intento de incremento con la clave fallo-1 devuelve HTTP 503. Complementa la Figura 38.

Figura 63

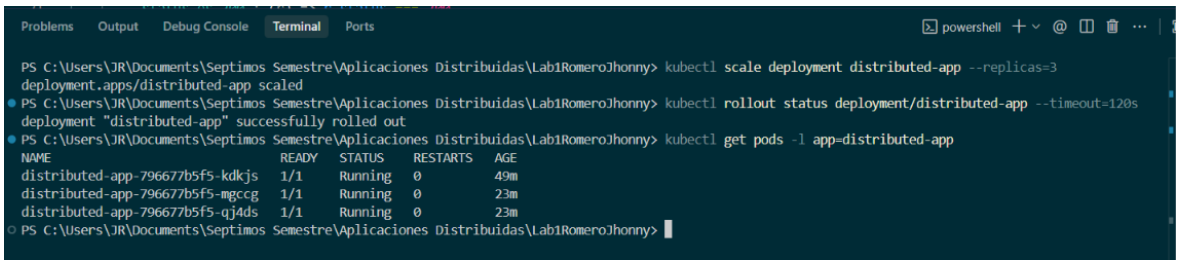
Coordinador escalado a cero y respuesta 503 (captura complementaria)



Se escaló nuevamente a 3 réplicas antes de conectar la aplicación al coordinador.

Figura 64

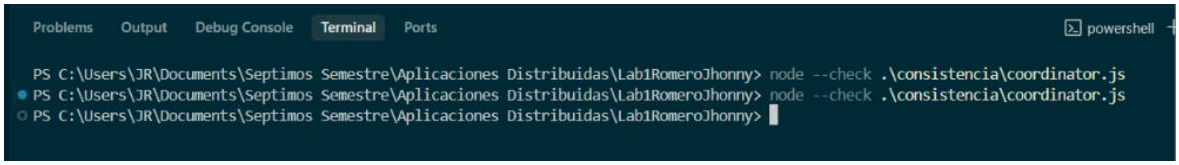
Regreso a tres réplicas para el experimento de consistencia



node --check sobre coordinator.js sin errores.

Figura 65

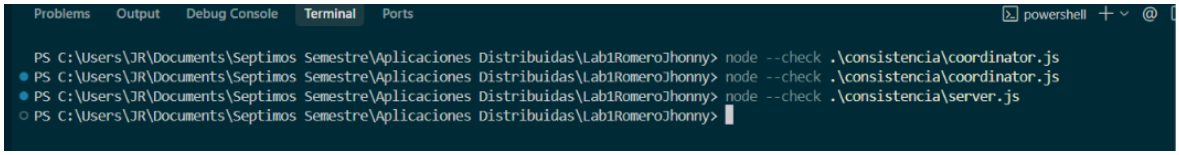
Verificación de sintaxis del coordinador



node --check sobre server.js (v4) sin errores.

Figura 66

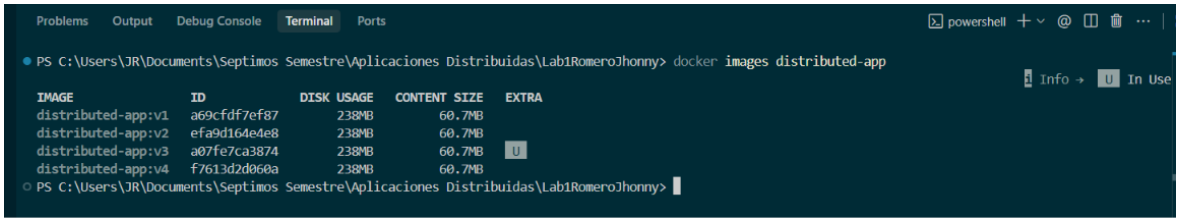
Verificación de sintaxis de la réplica v4



docker images muestra v1, v2, v3 y la nueva v4 disponibles localmente.

Figura 67

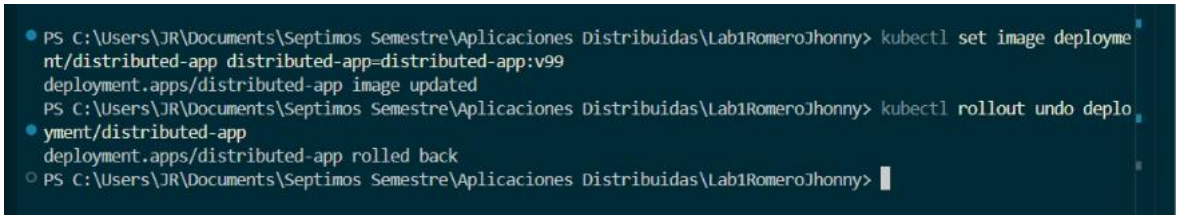
Imágenes locales tras construir v4



Secuencia breve de set image a v99 y rollout undo; versión condensada del fallo y recuperación del apartado «Fallo con imagen inexistente v99 y rollback».

Figura 68

Cambio de imagen y rollback (resumen en PowerShell)



Comprobación de winget v1.29.380, usado para instalar herramientas del entorno.

Figura 69

Entorno: winget

